WhatNext Vision Motors: Shaping the Future of Mobility with and Excellence

User Story:

As a sales representative at WhatsNext Vision Motors, I want an intelligent CRM system that enhances the customer ordering experience and streamlines operational processes.
I want the system to automatically suggest the nearest dealer location based on the customer's address so that customers can easily access the most convenient dealership and complete their orders efficiently.
I also want the system to validate vehicle stock availability and prevent customers from placing orders for out-of-stock vehicles, ensuring accurate order placement and avoiding customer dissatisfaction.
Additionally, I need the system to automatically update order statuses based on stock availability, marking orders as "Confirmed" when vehicles are in stock and "Pending" when they are not, so that customers receive clear and transparent updates.
Furthermore, I want automated processes to handle bulk order status updates, reducing manual effort and ensuring real-time accuracy in order tracking.
Lastly, I would like the system to improve overall efficiency by minimizing manual tasks, enabling staff to focus on strategic activities while delivering a faster, more reliable, and customer-friendly service.

1.INTRODUCTION

1.1 Project Overview :

WhatsNext Vision Motors is revolutionizing its customer experience and operational efficiency with a cutting-edge Salesforce CRM implementation. The project streamlines the vehicle ordering process by auto-assigning orders to the nearest dealer based on customer location and preventing orders for out-of-stock vehicles. Automated workflows update order statuses dynamically and send scheduled email reminders for test drives. Key technical implementations include Apex triggers for stock validation, batch jobs for stock updates, and scheduled Apex for automated order processing. This initiative enhances customer satisfaction, improves order accuracy, and boosts overall operational efficiency.

1.2 Purpose :

● Streamline the vehicle ordering process by providing a seamless and user-friendly system for customers
● Automatically suggest the nearest dealer location based on customer address to improve convenience and accessibility
● Ensure accurate order placement by validating vehicle stock availability and preventing out-of-stock orders
● Automate order status updates (Confirmed/Pending) based on real-time stock conditions
● Improve data accuracy and transparency in order management and tracking
● Enhance customer satisfaction by providing clear communication regarding order status
● Reduce manual effort and administrative workload through process automation
● Enable efficient decision-making using real-time data and system insights

2. IDEATION PHASE

2.1 Problem Statement (Customer Problem Statement Template)

PS	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	Sales Representati ve	Process vehicle orders efficiently	Orders are delayed or incorrect	Stock availability and order status are checked manually	Stressed and inefficient
PS-2	Store Manager	Monitor orders and stock availabilit y	Order status is unclear and not updated in real time	Data is not synchronize d and updates are manual	Confused and dissatisfied
PS-3	Customer	Place vehicle orders easily and find nearest dealer	I may select a far dealer or order unavailable vehicles	There is no system to suggest dealers or validate stock	Frustrated and inconvenienc ed

2.2 Empathy Map Canvas

Target Users

- Sales Representatives
- Managers
- System Administrators
- Business Analysts

Empathy Analysis

Aspect	Description
Think & Feel	Need an efficient system to manage vehicle orders, track stock availability, and ensure accurate order processing
Hear	Feedback from customers about order delays, stock issues, and dealer accessibility
See	Multiple Disconnected systems, manual tracking of stock, and lack of real-time order updates and spreadsheets
Say & Do	Manually check stock, coordinate with dealers, and update order status
Pain	Order errors, delays in processing, lack of stock visibility, and customer dissatisfaction
Gain	Automated dealer suggestions, real-time stock validation, accurate order status updates, and improved efficiency

2.3 Brainstorming

Brainstormed Ideas

- Manual order management system
- Excel-based vehicle and stock tracking
- Web-based vehicle ordering system
- Salesforce-based CRM solution
- Automated dealer suggestion system
- Real-time stock validation system
- Automated order status update system

Selected Idea:

3. **Salesforce-based CRM Application for WhatsNext Vision Motors with intelligent dealer suggestion, stock validation, and automated order management system.**

REQUIREMENT ANALYSIS PHASE

3.1 Customer Journey Map

Journey Stages:

- Login to CRM System
- Access Vehicle Ordering Module
- Enter Customer Details
- System Suggests Nearest Dealer
- Select Vehicle and Check Availability
- Place Order
- System Validates Stock
- Order Status Update (Confirmed / Pending)
- View Order Details
- Decision Making & Follow-up

3.2 Solution Requirements

Functional Requirements

FR No	Requirement
FR-1	User Login & Authentication
FR-2	Vehicle, Dealer, and Customer Object Creation
FR-3	Customer Data Entry & Management
FR-4	Automatic Dealer Suggestion based on Customer Address
FR-5	Vehicle Stock Validation before Order Placement
FR-6	Vehicle Order Creation & Management
FR-7	Automated Order Status Update (Pending / Confirmed)
FR-8	Test Drive Booking Management
FR-9	Vehicle Service Request Management
FR-10	Report Generation for Orders, Customers, and Vehicles
FR-11	Dashboard Visualization for business insights

Non-Functional Requirements

NFR-1: Usability

The system should be easy to use for sales representatives, managers, administrators, and analysts without requiring technical training.

Requirements :

- Simple and intuitive user interface
- Clear navigation menus
- Easy-to-understand forms

Example:

A sales representative should be able to place a vehicle order within a few minutes.

NFR-2: Security

The system must protect sensitive customer and business data.

Requirements:

- Role-based access control
- Secure login authentication
- Data encryption
- Field-level security

Examples:

- Profiles and permission sets
- Object-level and field-level security
- Restricted billing access
- Secure login and password policies

NFR-3: Reliability

The CRM system should work correctly at all times and store data accurately without loss.

Requirements:

- Automatic data saving
- Regular system backups
- Error handling mechanisms
- Stable cloud environment

Example :

Order data should not be lost even if the system disconnects.

Salesforce cloud ensures automatic backups.

NFR-4: Performance

The system should respond quickly and handle large amounts of data efficiently.

Requirements:

- Fast page loading
- Quick order processing
- Efficient report generation

Example: Order confirmation should happen within seconds after validation.

Optimized queries and caching improve speed.

NFR-5: Availability

The system should be accessible anytime.

Requirements:

- 24/7 system availability
- Minimal downtime
- Cloud-based hosting
- Disaster recovery

Example: Managers should be able to access the CRM system during peak business hours, weekends, or late nights.
Users should access the system anytime from any location.

NFR-6: Scalability

The system should support growth.

Requirements:

- Handle increasing customers and orders
- Expandable storage
- Maintain performance

Example: If TheChaiShop&Co expands to multiple branches, the CRM system should support thousands of customers without slowing down.
Salesforce infrastructure automatically scales.
3.3 System should support expansion to multiple cities and dealers.

Data Flow Diagram

1.1 User → Salesforce UI → Custom Objects → Validation Rules → Flows → Reports

Technology Stack

Layer	**Technology**
Frontend	Salesforce Lightning
Database	Salesforce Cloud
Automation	Flows
Backend	Salesforce Platform

2. PROJECT DESIGN PHASE

2.1 Problem–Solution Fit

The system replaces manual and disconnected vehicle ordering processes with an automated and intelligent CRM solution. By integrating dealer location mapping, stock validation, and automated order management, the system eliminates errors and inefficiencies in order processing.

The automatic dealer suggestion feature ensures customers are directed to the nearest dealer, improving convenience and reducing effort. Real-time stock validation prevents orders for unavailable vehicles, avoiding confusion and enhancing customer satisfaction.

Automated order status updates (Pending/Confirmed) provide clear and transparent communication to customers. Additionally, cloud-based storage ensures data accuracy, prevents data loss, and enables real-time access to information.

Overall, the system enhances operational efficiency, improves decision-making through accurate data, and delivers a faster, more reliable, and customer-friendly vehicle ordering experience.

2.2 Proposed Solution

Functional Requirements

Parameter	Description
User Login & Authentication	Allows authorized users to securely log in to the system using valid credentials and access role-based features.
Customer Record Management	Enables users to create, view, update, and manage customer details such as name, contact information, and address.
Dealer Management	Allows management of authorized vehicle dealers including location, contact details, and dealer information.
Vehicle Management	Enables storage and management of vehicle details such as model, price, stock quantity, and availability status.
Order Management	Supports creation and tracking of vehicle orders, including customer, vehicle selection, and order date.
Nearest Dealer Suggestion	Automatically suggests the nearest dealer based on the customer's address to improve convenience.
Stock Validation System	Prevents customers from placing orders for vehicles that are out of stock, ensuring accurate order processing.
Automated Order Status Update	Automatically updates order status to "Confirmed" or "Pending" based on vehicle stock availability.
Test Drive Management	Allows customers to book and manage vehicle test drives with status tracking.
Service Request Management	Enables customers to raise vehicle service requests and track their status.
Validation Rules	Ensures accurate data entry by validating fields such as phone number and required details.
Validation Rules	Provides different record types and layouts for better data organization and user experience.
Record Types & Page Layouts	Maintains standardized values for fields such as vehicle type, order status, and service status.
Global Picklists	Automates tasks such as dealer suggestion, order updates, and notifications using Salesforce Flows.
Workflow & Flow Automation	Generates detailed reports on orders, vehicles, customers, and overall performance.

Report Generation	**Displays real-time analytics through interactive dashboards for better decision-making.**
Dashboard Visualization	**Ensures secure storage and recovery of data using Salesforce cloud infrastructure.**
Data Backup & Recovery	**Controls system access using profiles and permission sets for different user roles.**

2.3 Solution Architecture

Architecture Layers

1. Presentation Layer – Lightning UI
2. Application Layer – Flows and Logic
3. Data Layer – Custom Objects
4. Analytics Layer – Reports and Dashboards

This layered architecture ensures modularity, scalability, and maintainability.

3. PROJECT PLANNING & SCHEDULING

3.1 Project Planning

● The project followed an Agile-based development approach with clearly defined milestones and sprints.
● A Salesforce Developer account was created to set up and configure the development environment.
● Custom objects were designed to manage vehicle, dealer, customer, order, test drive, and service request data.
● Custom tabs and a Lightning Application were developed for easy navigation across all modules.
● Fields, relationships, and global picklists were configured to maintain data consistency and standardization.
● Page layouts and record types were created to organize data based on different processes such as orders and services.
● Validation rules were implemented to ensure accurate data entry and prevent invalid records.
● Automation using Salesforce Flows was developed for dealer suggestion, stock validation, and order status updates.
● Apex and Batch Classes were implemented to handle bulk order processing and scheduled status updates.

● Reports and dashboards were created to monitor vehicle orders, stock availability, and overall performance.
● Regular testing and sprint reviews were conducted to ensure smooth functionality and fix issues.
● The project was completed within the planned timeline through structured planning and execution.**Product Backlog, Sprint Schedule, and Estimation (4 Marks)**

4. **Project Development Phase:**

Project Flow

Milestone 1: Creation of Salesforce Developer Account
Milestone 2: Creation of Custom Objects (Vehicle, Dealer, Customer, Order, Test Drive, Service Request)

Milestone 3: Creation of Custom Tabs for all objects
Milestone 4: Development of Lightning Application for system navigation

Milestone 5: Fieldss and relationship

Milestone 6: Automation

Milestone 7: Ceation of apex and trigger jobs
Milestone 8: Conclution

What you'll learn

1. Real Time Salesforce Project
2. Standard and Custom objects
3. Picklist Value sets
4. Types of Fields & their relationships.
5. Page Layouts
6. Record Types
7. Validation Rules
8. Email Templates
9. Flows
10. Reports and Dashboards.

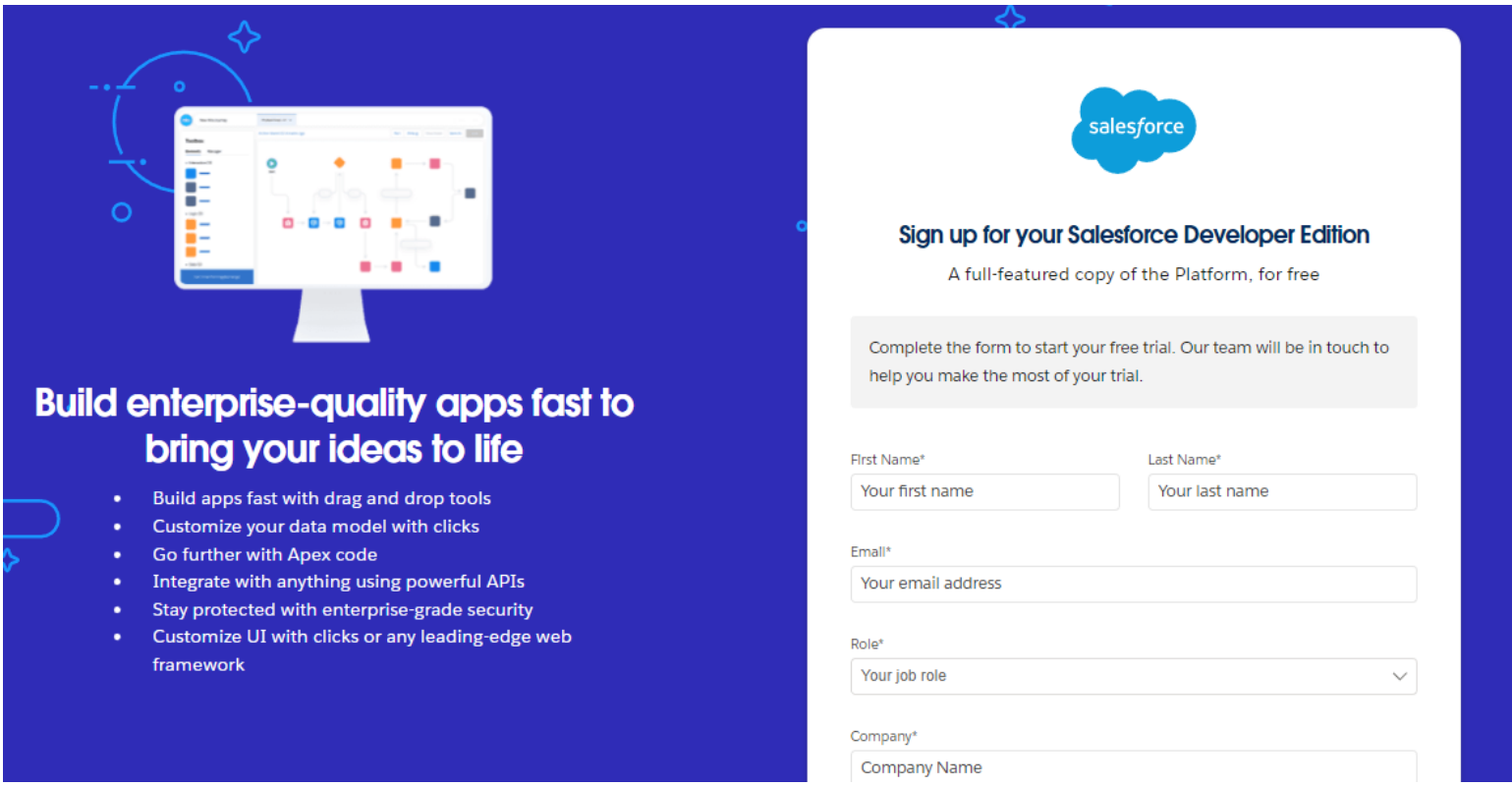
Implementation

Milestone 1 - Salesforce developer account creation :

Activity 1: Creating Developer Account:

Creating a developer org in salesforce.

1. Go to https://developer.salesforce.com/signup
2. On the sign up form, enter the following details :



1) First name & Last name
2) Email
3) Role : Developer
4) Company : College or Company Name
5) County : India

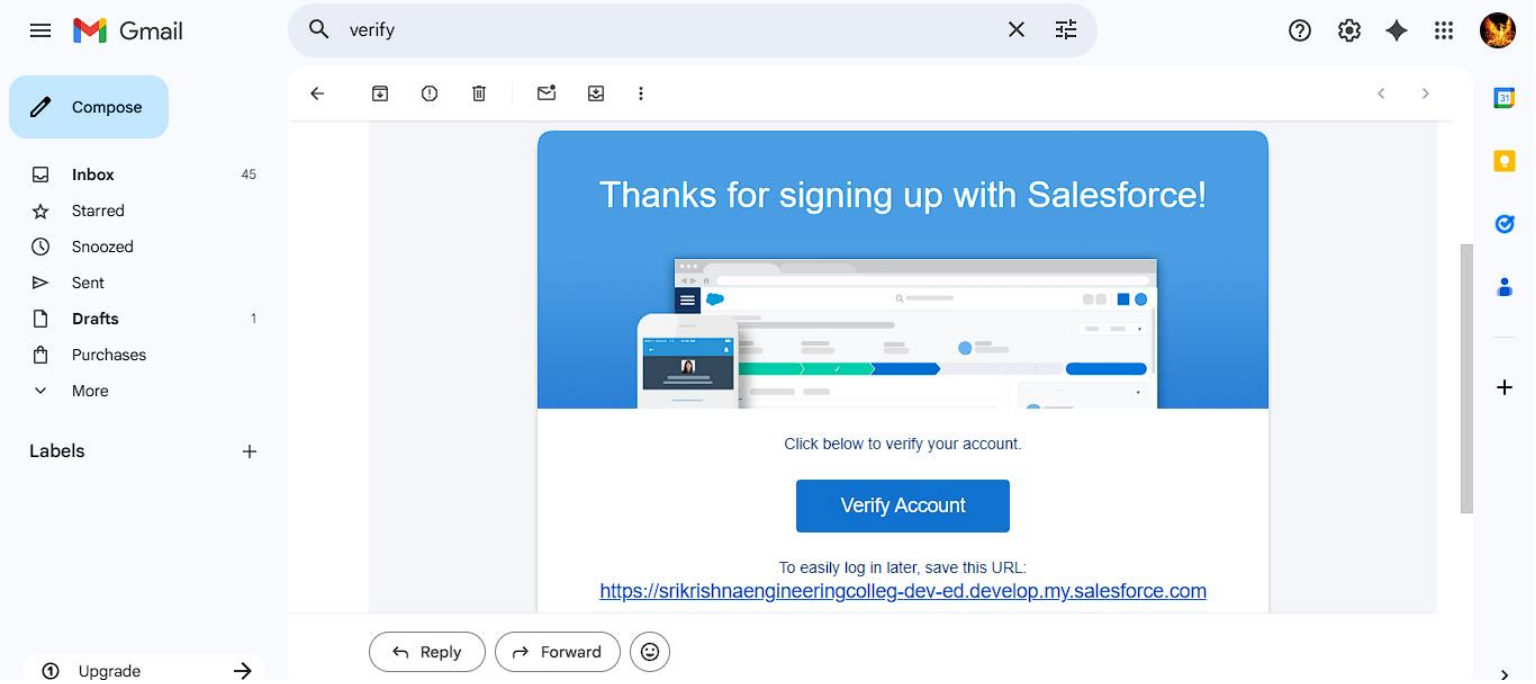
6) Postal Code : pin code
7) Username : should be a combination of your name and company

This need not be an actual email id, you can give anything in that format : username@organization.com

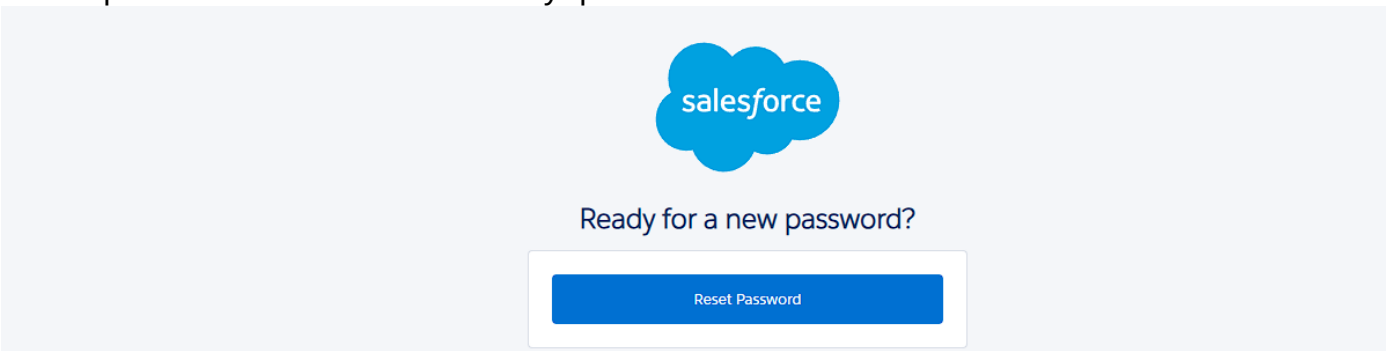
Click on sign me up after filling these.

Activity 2: Account Activation :

1. Go to the inbox of the email that you used while signing up. Click on the verify account to activate your account. The email may take 5-10mins.

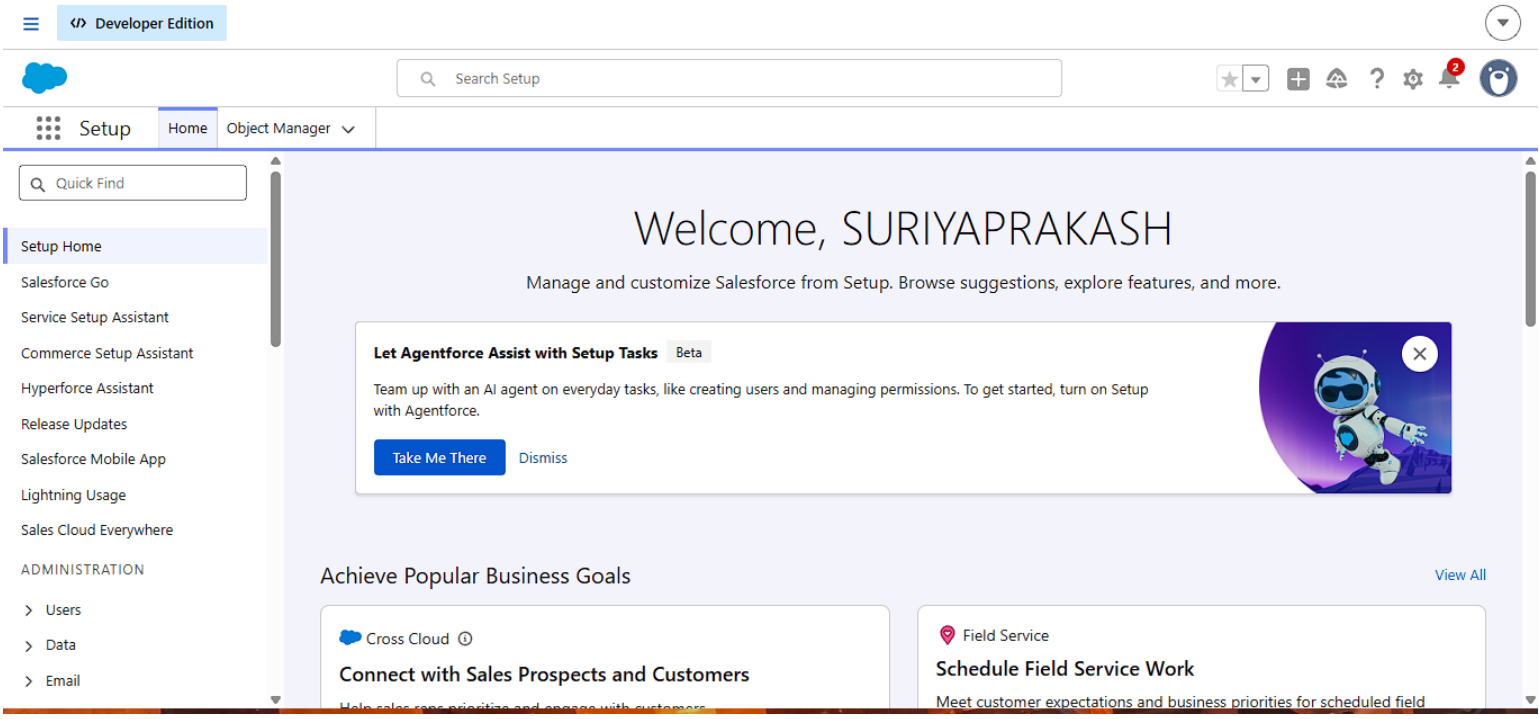


2. Click on Verify Account.
3. Give a password and answer a security question and click



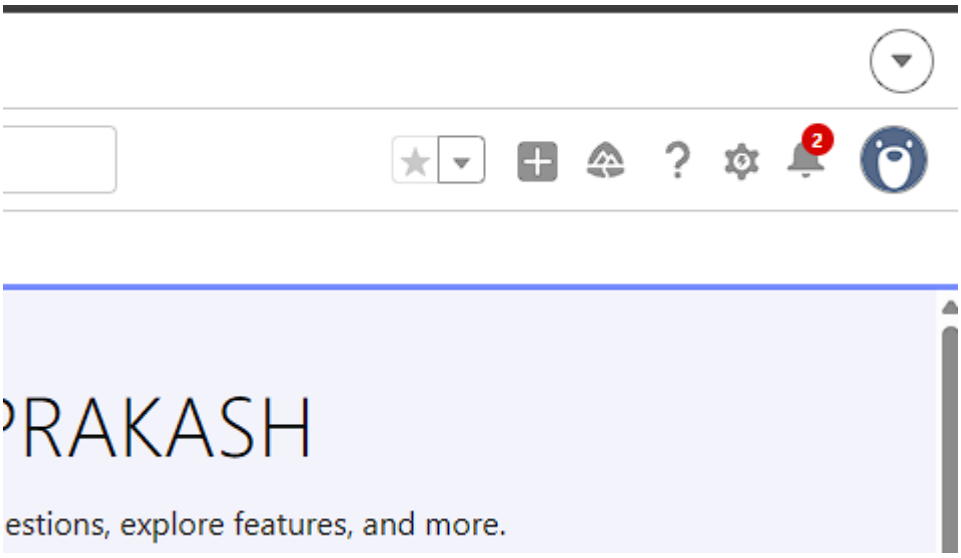
on change password.

4. Then you will redirect to your salesforce setup page.

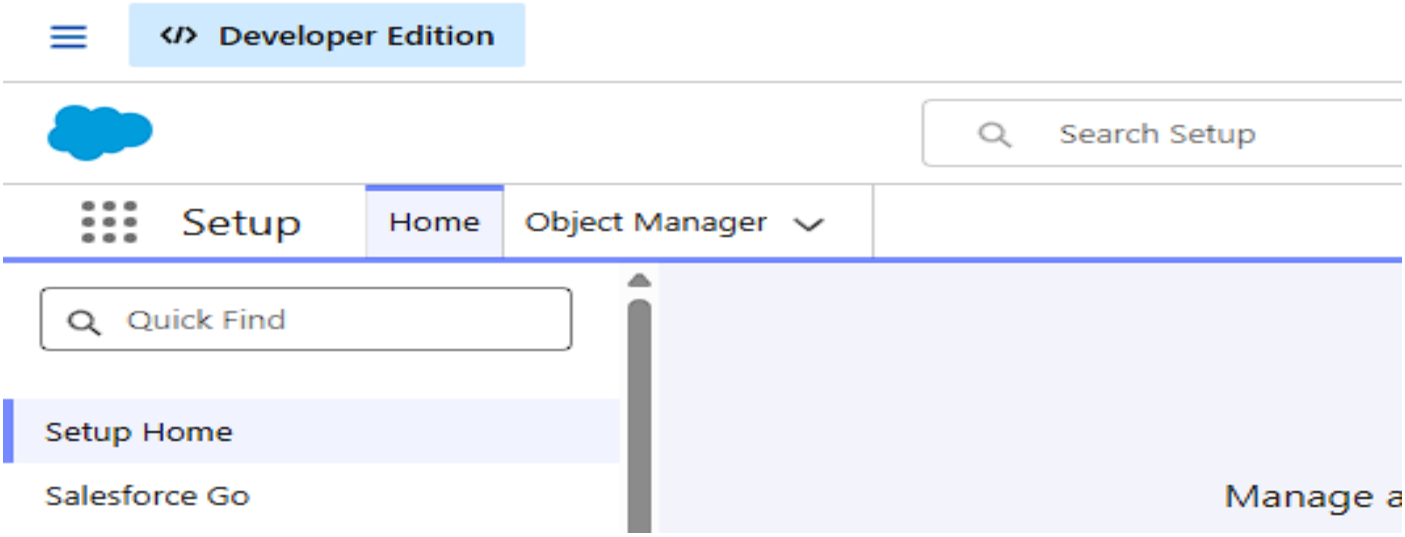


Milestone 2 - Objects

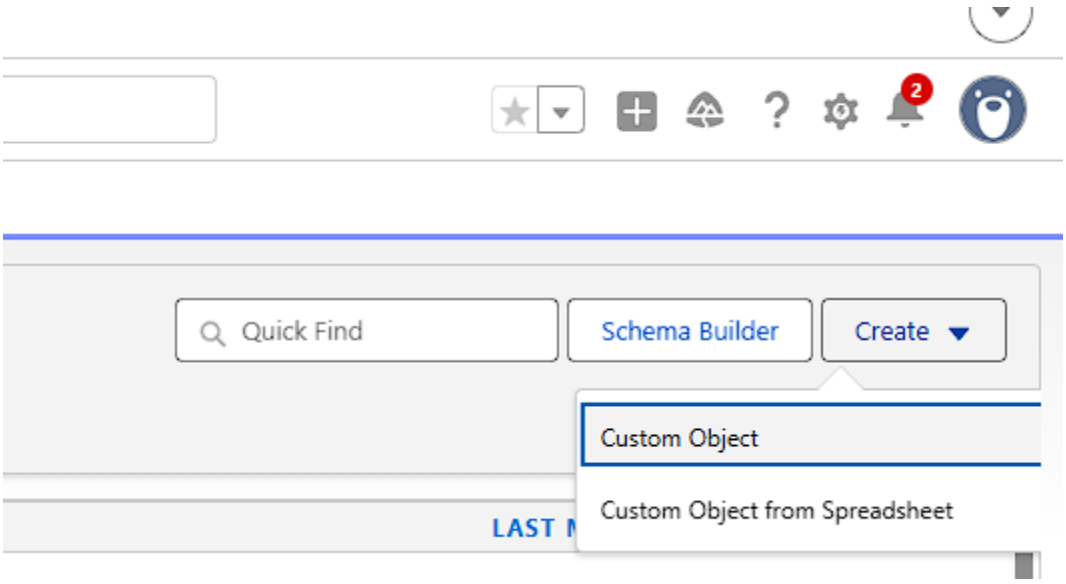
1. In your Salesforce org, click gear icon on the top left and select Setup to open Setup.



2. Click the Object Manager tab. If you don't see it, enter Object Manager in the Quick Find box.



3.On the Object Manager page, click Create | Custom Object.



Activity 1 : Vehicle Account Object

1. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.
   - Enter the label name→ Vehicle
   - Plural label name→ Vehicles
   - Enter Record Name Label and Format
   - Record Name → Vehicle Name

- Data Type → Text
- Click on Allow reports,
- Allow search → **Save.**

Activity 2 : Create Dealer Object

1. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.
    - Enter the label name→ Vehicle Dealer
    - Plural label name→ Vehicle Dealers
    - Enter Record Name Label and Format
    - Record Name → Dealer Name
    - Data Type → Text
    - Click on Allow reports,
    - Allow search → **Save**

Activity 3 : Enterprise Account Object

1. From the setup page → Click on Object Manager → Click on Create → Click on Custom Object.
    - **Label** : Enterprise Account
    - **Plural Label** : Enterprise Accounts
    - Enter Record Name Label and Format
        Record Name : Enterprise Account Name
        Data Type **:** Text
    - **In Optional features :** check the boxes for Allow Reports | Allow Activities | Track Field History.
    - **Search Status :** check the box for Allow Search.
    - Save

Activity 4 :Repeat the Same steps for remaining fields

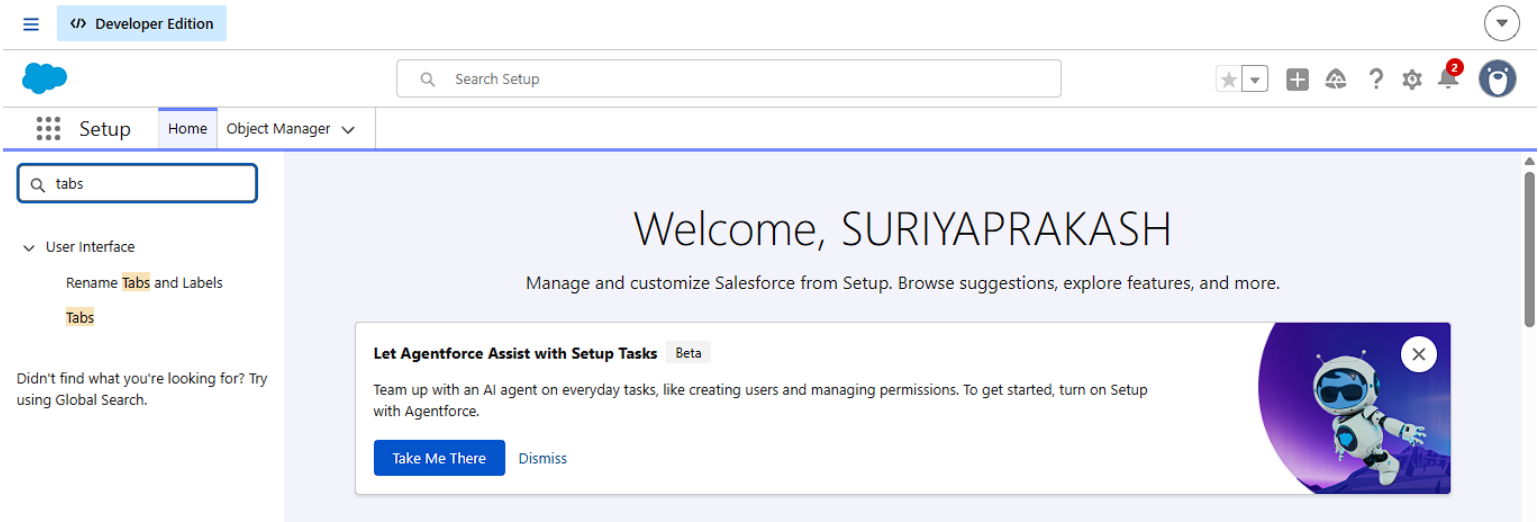
Object Name	Purpose	Relationships
Vehicle__c	Stores vehicle details	Related to Dealer & Orders
Vehicle_Dealer__c	Stores authorized dealer info	Related to Orders
Vehicle_Customer__c	Stores customer details	Related to Orders & Test Drives
Vehicle_Order__c	Tracks vehicle purchases	Related to Customer & Vehicle
Vehicle_Test_Drive__c	Tracks test drive bookings	Related to Customer & Vehicle
Vehicle_Service_Request__c	Tracks vehicle servicing requests	Related to Customer &

		Vehicle

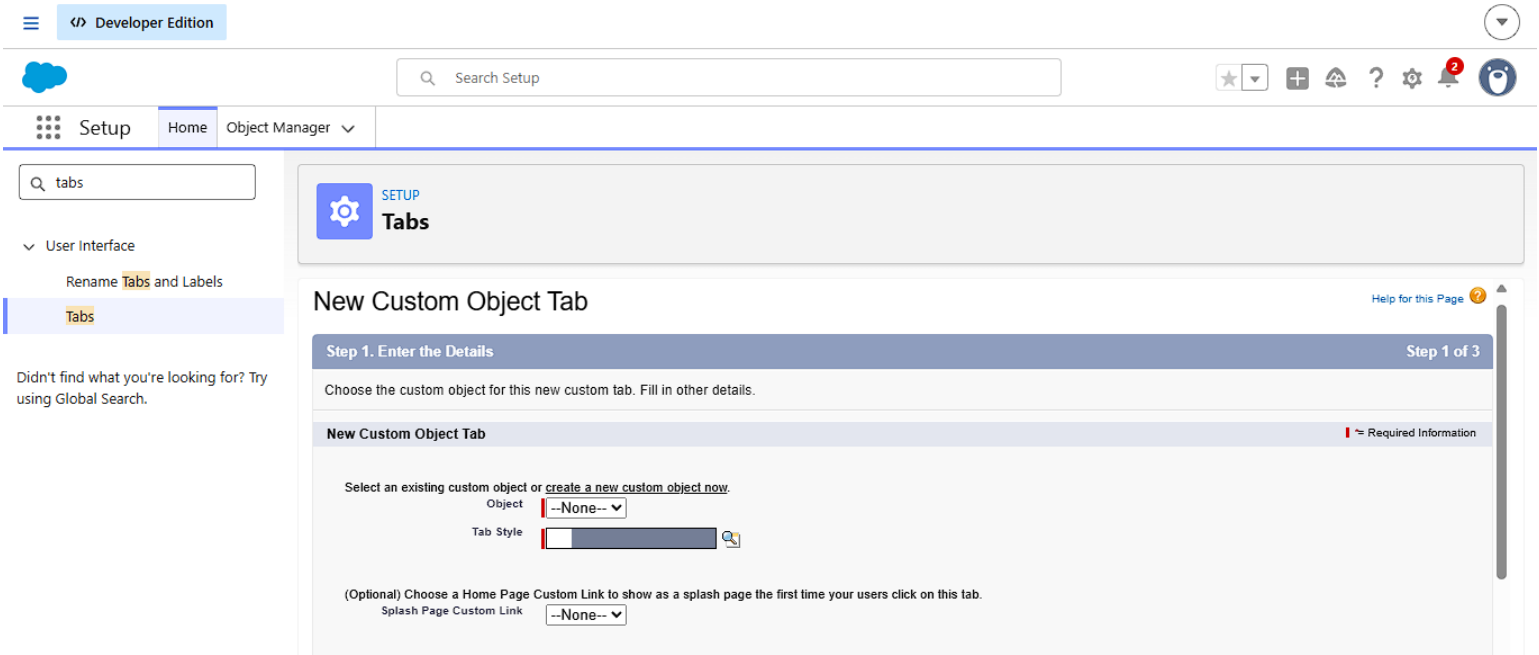
Milestone 3 - Tabs

Activity 1 : Custom tabs creation

1. Go to setup page → type Tabs in Quick Find bar → click on tabs → New (under custom object tab)



1. Select Object(Vehicle) → Select any tab style → Next (Add to profiles page) keep it as default → Next (Add to Custom App) keep it as default → Save.



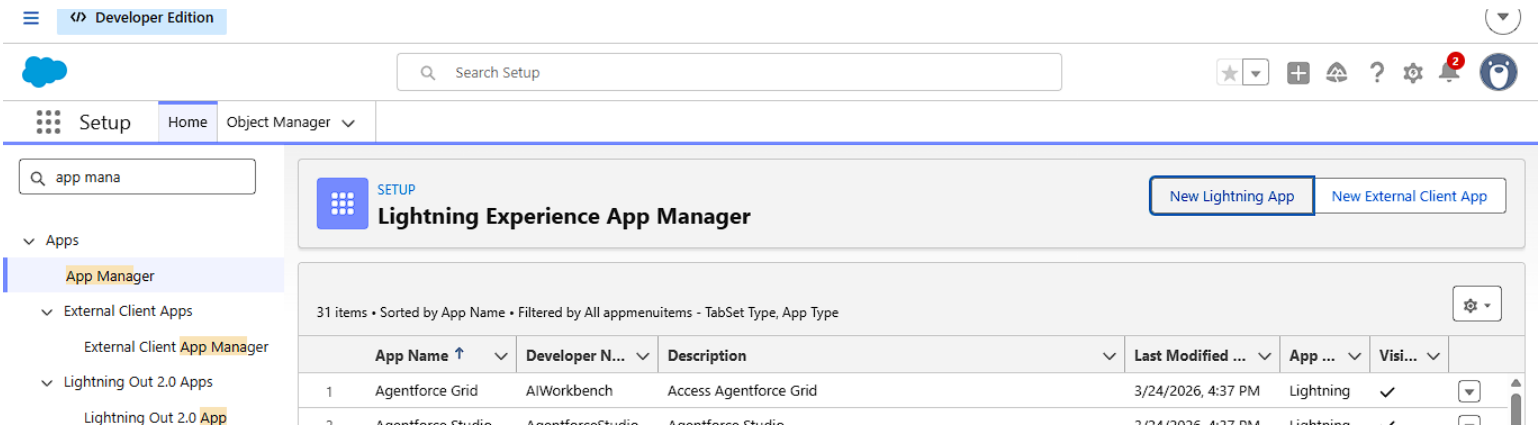
2. Repeat the same steps for Vehicle Dealer, Vehicle Customer , Vehicle Order and Vehicle Test

Drive.

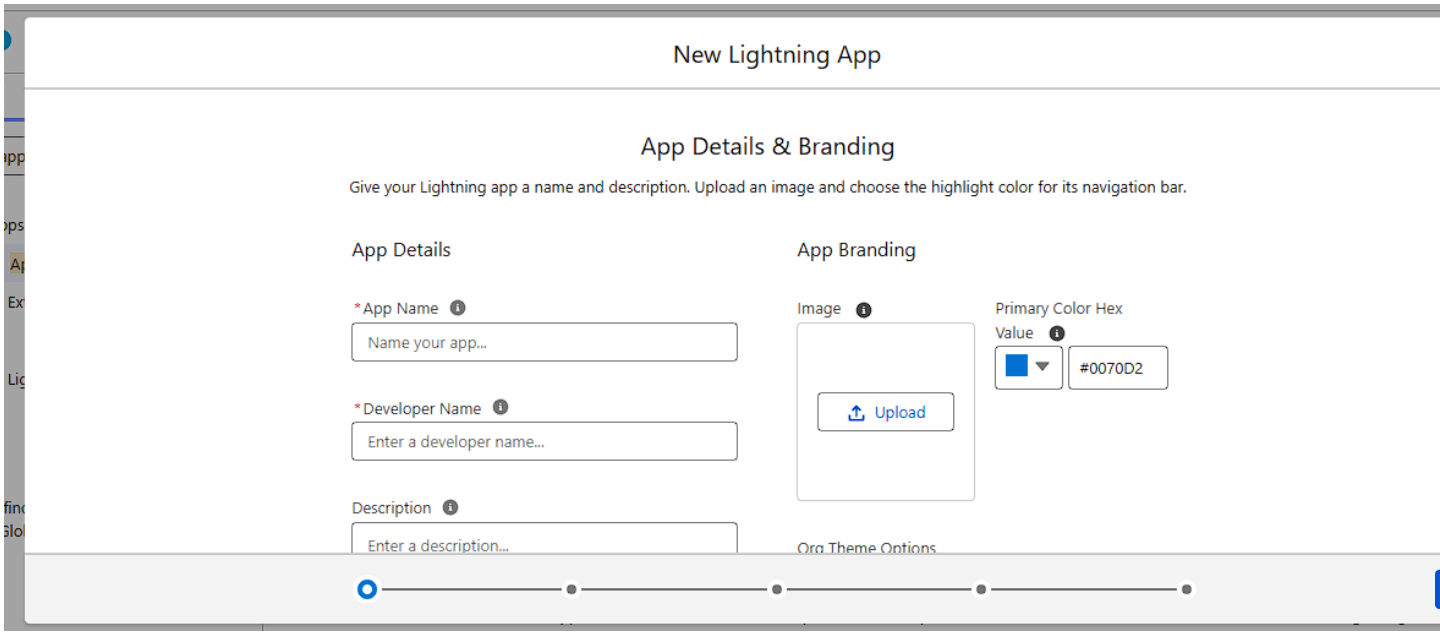
Milestone 4 - The Lightning App

Activity 1 : Create a lightning app

1. Go to setup page → search “app manager” in quick find → select “app manager” → click on New lightning App.



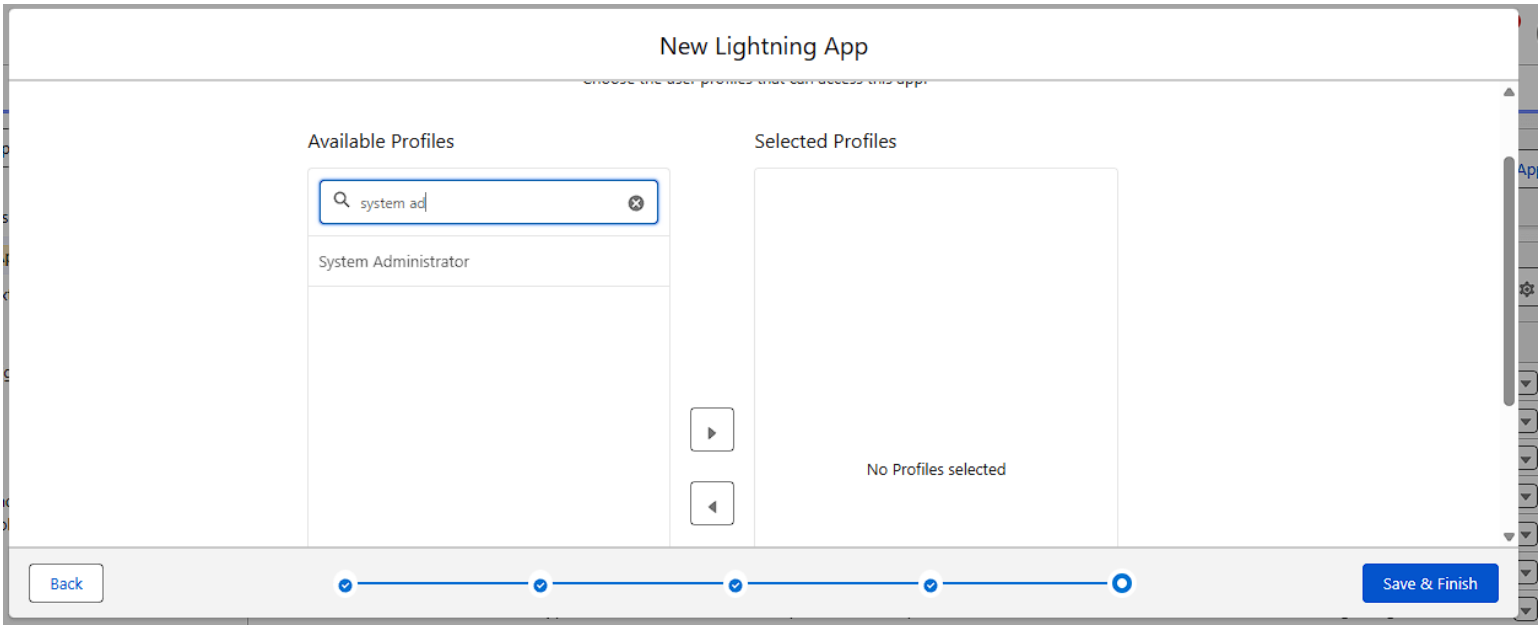
- Fill the app name in app details and branding as follow
- App Name : WhatNext Vision Motors
- Developer Name : this will auto populated
- Description : Give a meaningful description
- Image : optional (if you want to give any image you can otherwise not mandatory)
  Primary color hex value : keep this default
- Then click Next → (App option page) keep it as default → Next → (Utility Items) keep it as default → Next.



To Add Navigation Items:

```
app = Flask(__name__)
```

Search the items in the search bar(Vehicle, Dealer, Customer, Order, Test Drive, Service Request, Reports, Dashboard) from the search bar and move it using the arrow button → Next.
Note: select the custom object which we have created in the previous activity.
To Add User Profiles:

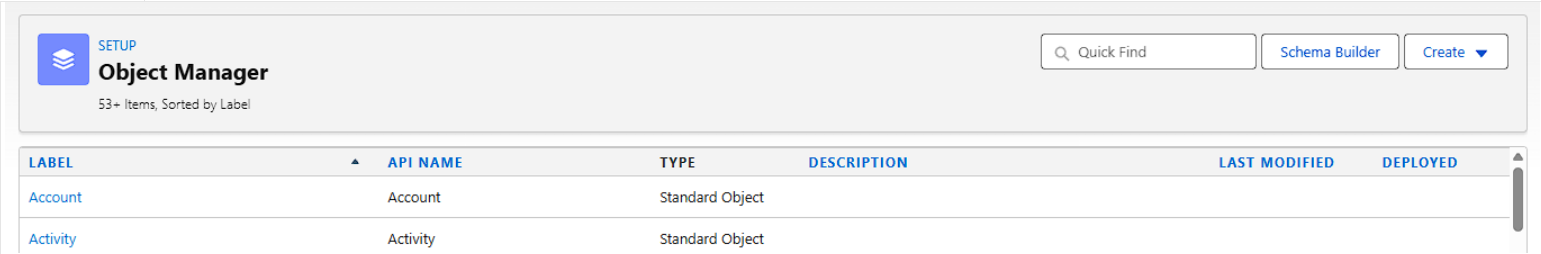


2. Click Save & Finish.

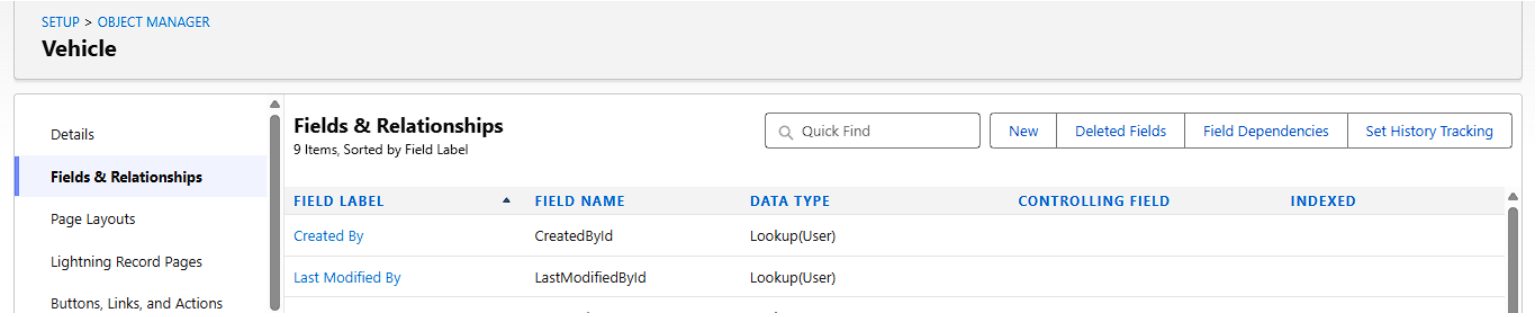
Milestone 5: Fields & Relationships

Activity 1 : Create Custom fields for Vehicle Object

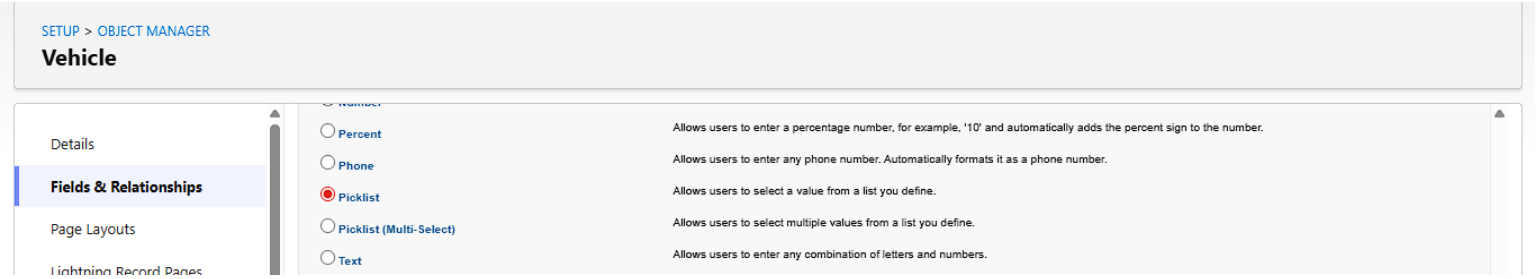
1. Go to setup → click on Object Manager → type object name(Vehicle) in quick find bar→ click on the object.



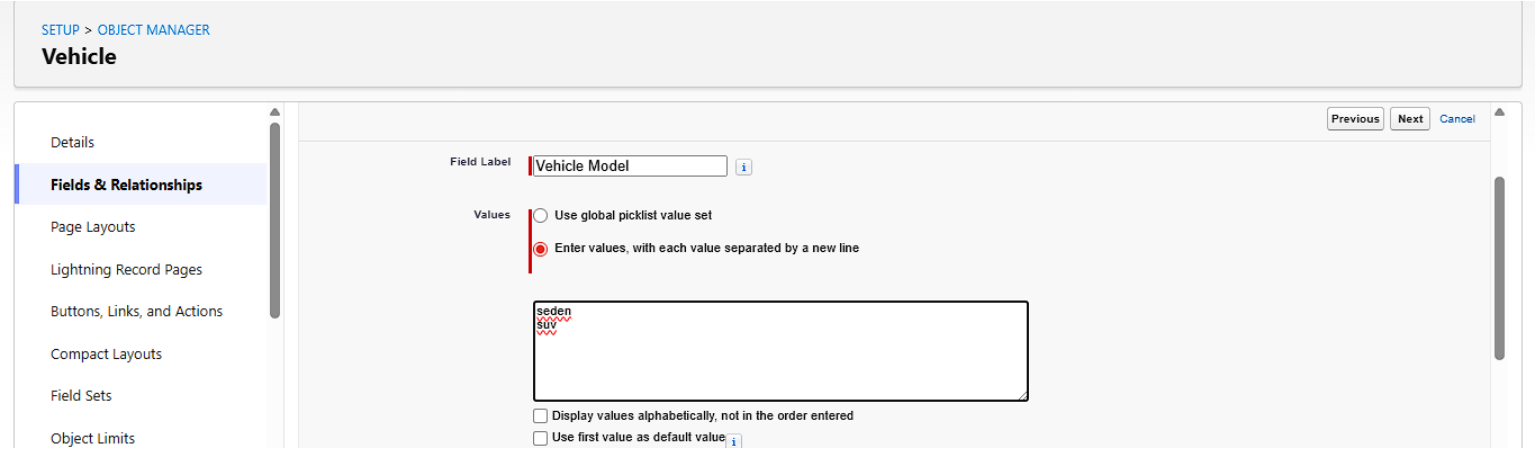
2. Now click on “Fields & Relationships” → New



3. Select Data type as “PickList”



4. Click on next.



Fill the above as following:
Field Label: Vehicle Model
Fill PickList Values

Click on Next → Next → Save and new.

Activity 2: Create Custom fields for remaining objects

1. Vehicle__c (Custom Object)

Vehicle_Name__c (Text)
Vehicle_Model__c (Picklist: Sedan, SUV, EV, etc.)
Stock_Quantity__c (Number)
Price__c (Currency)
Dealer__c (Lookup to Dealer__c)
Status__c (Picklist: Available, Out of Stock, Discontinued)

2.Vehicle_ Dealer__c (Custom Object)

Dealer_Name__c (Text)
Dealer_Location__c (Text)
Dealer_Code__c (Auto Number)
Phone__c (Phone)
Email__c (Email)

3. Vehicle_Order__c (Custom Object)

Customer__c (Lookup to Customer__c)
Vehicle__c (Lookup to Vehicle__c)
Order_Date__c (Date)
Status__c (Picklist: Pending, Confirmed, Delivered, Canceled)

4. Vehicle_Customer__c (Custom Object)

Customer_Name__c (Text)
Email__c (Email)
Phone__c (Phone)
Address__c (Text)
Preferred_Vehicle_Type__c (Picklist: Sedan, SUV, EV, etc.)

5. Vehicle_Test_Drive__c (Custom Object)

Customer__c (Lookup to Customer__c)
Vehicle__c (Lookup to Vehicle__c)
Test_Drive_Date__c (Date)
Status__c (Picklist: Scheduled, Completed, Canceled)

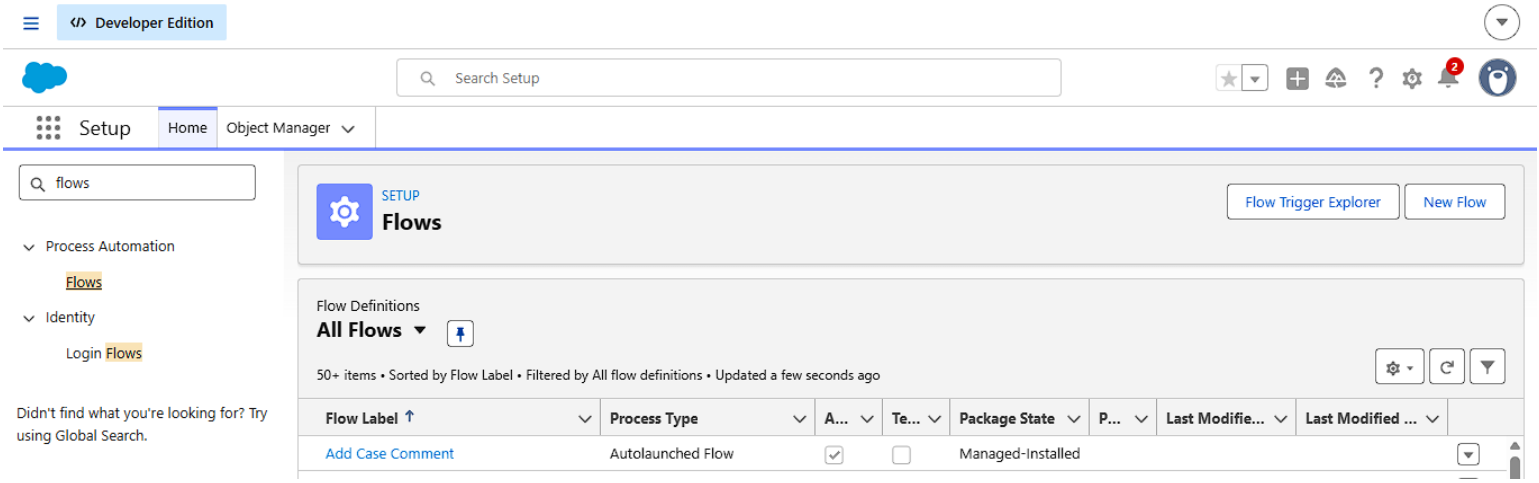
6. Vehicle_Service_Request__c (Custom Object)

Customer__c (Lookup to Customer__c)
Vehicle__c (Lookup to Vehicle__c)
Service_Date__c (Date)
Issue_Description__c (Text)
Status__c (Picklist: Requested, In Progress, Completed)

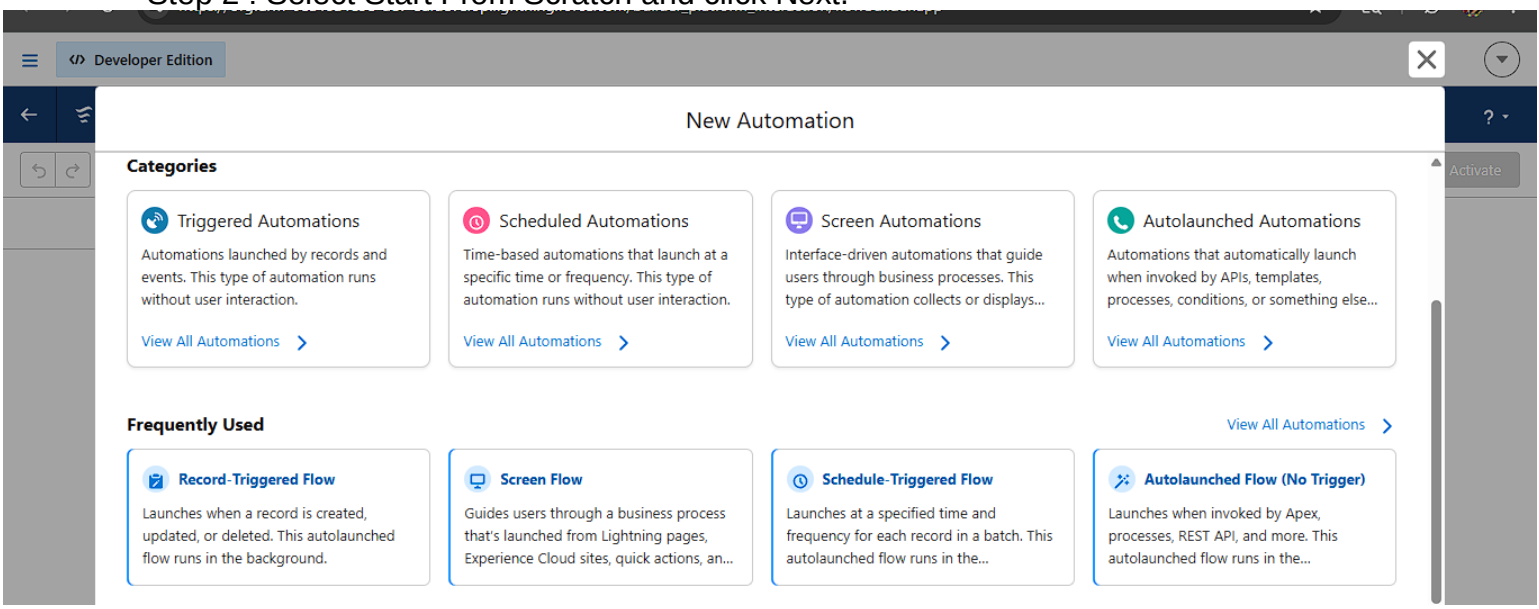
Milestone 6: Automation

Activity 1: Flow Creation

Step 1 : In Quick Find, type Flows and click on Flows. Click New Flow.



Step 2 : Select Start From Scratch and click Next.



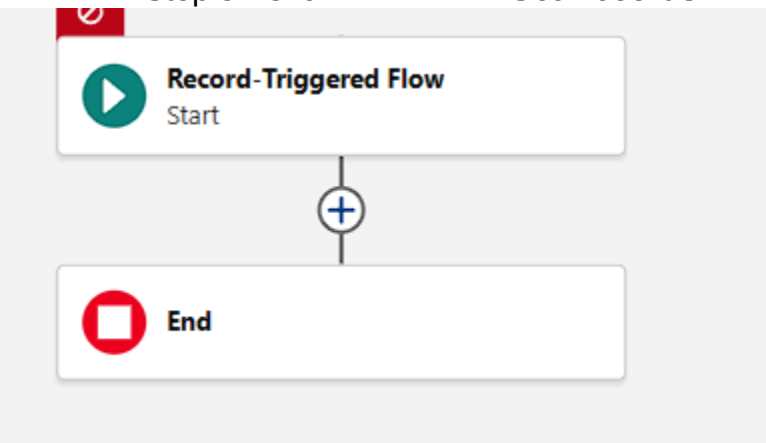
Step 3 : Select Record-Triggered Flow and click Create

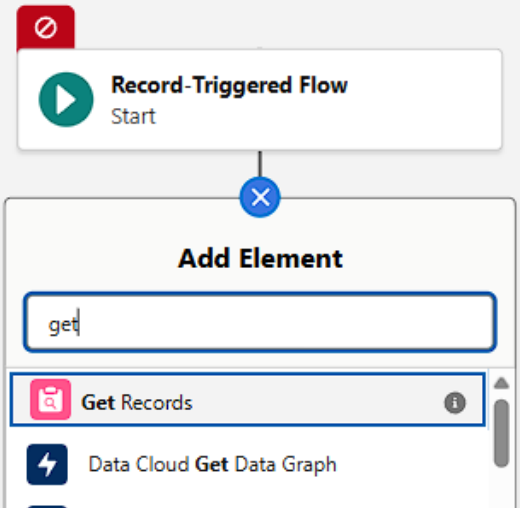
Step 4 : Select Vehicle Order Object
Trigger the Flow When : Select Record is Created.
Set Entry Condition :

All Conditions Are Met (AND)
Filed : Status__c
Operator : Equals
Value : Pending

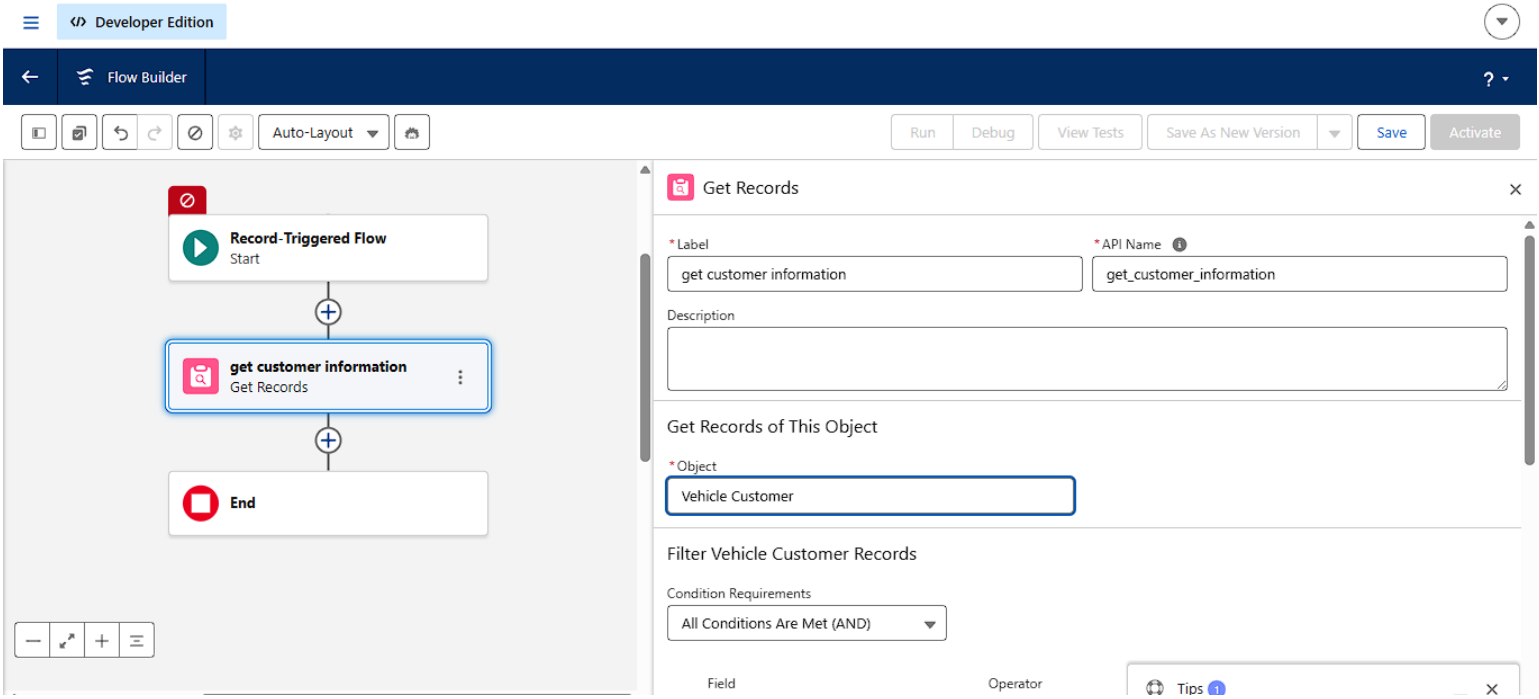
```
app = Flask(__name__)
```

Step 5 : Click **+** → Select **Get Records**

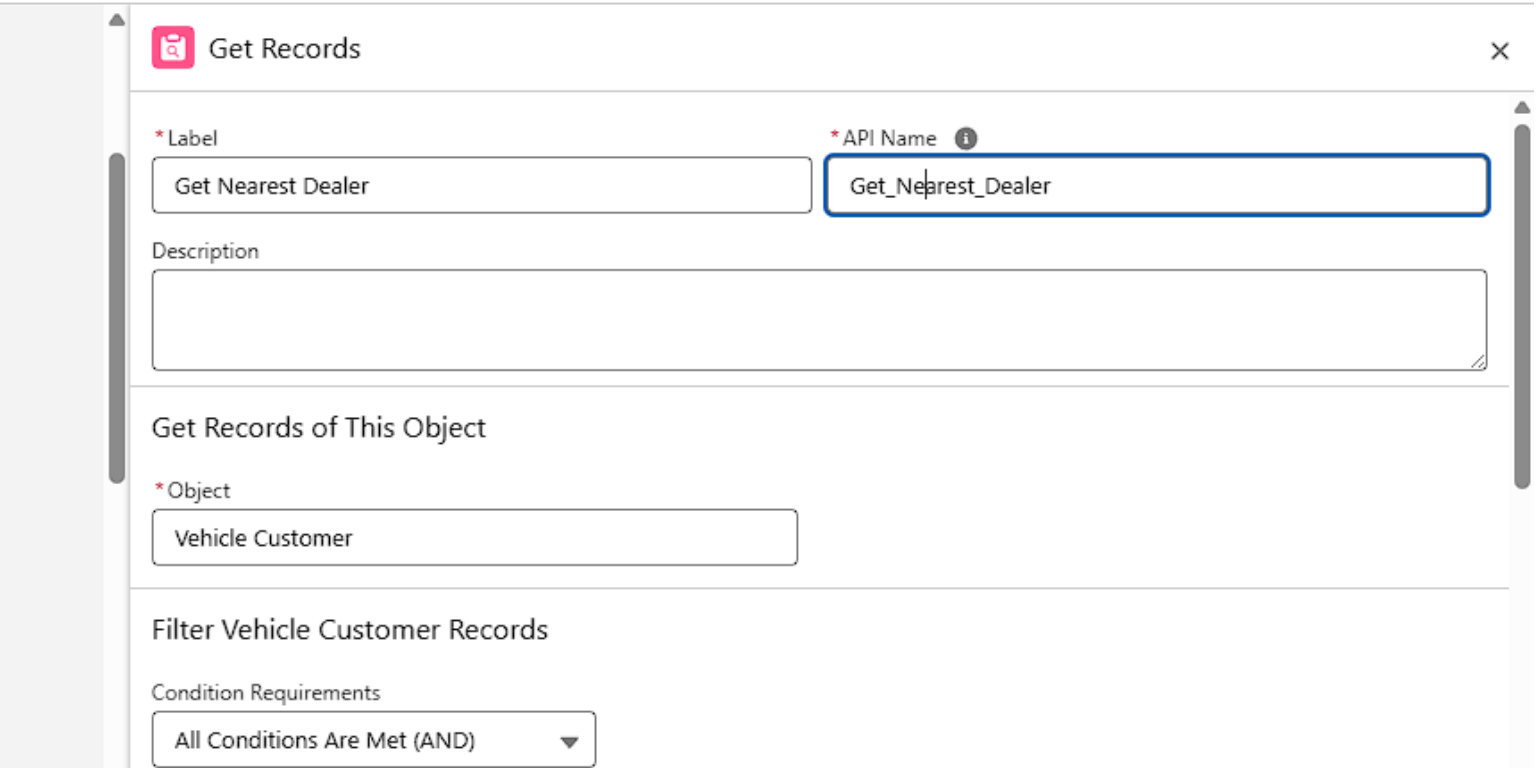




Step 6 : **Label :** Get Customer Information
Object : Vehicle Customer
Condition :
Field : Id
Operator : Equals
Value : {!$Record.Vehicle_Customer__c}

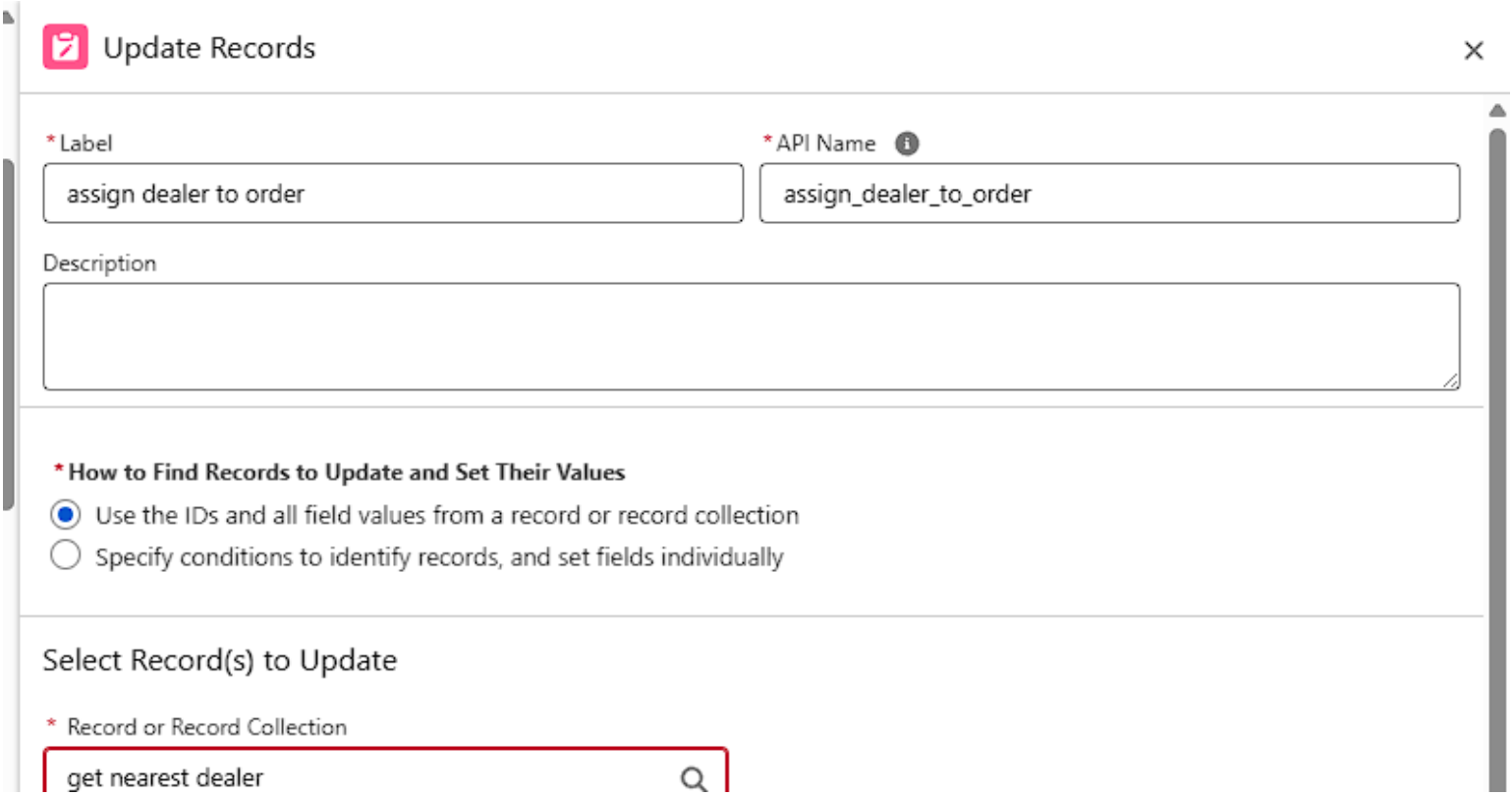


Step 7 : Click **+** → Select **Get Records**
Label : Get Nearest Dealer
Object : Vehicle Dealer
Condition :
Field : Dealer_Location__c
Operator : Equals
Value : {!Get_Customer_Information.Address__c}

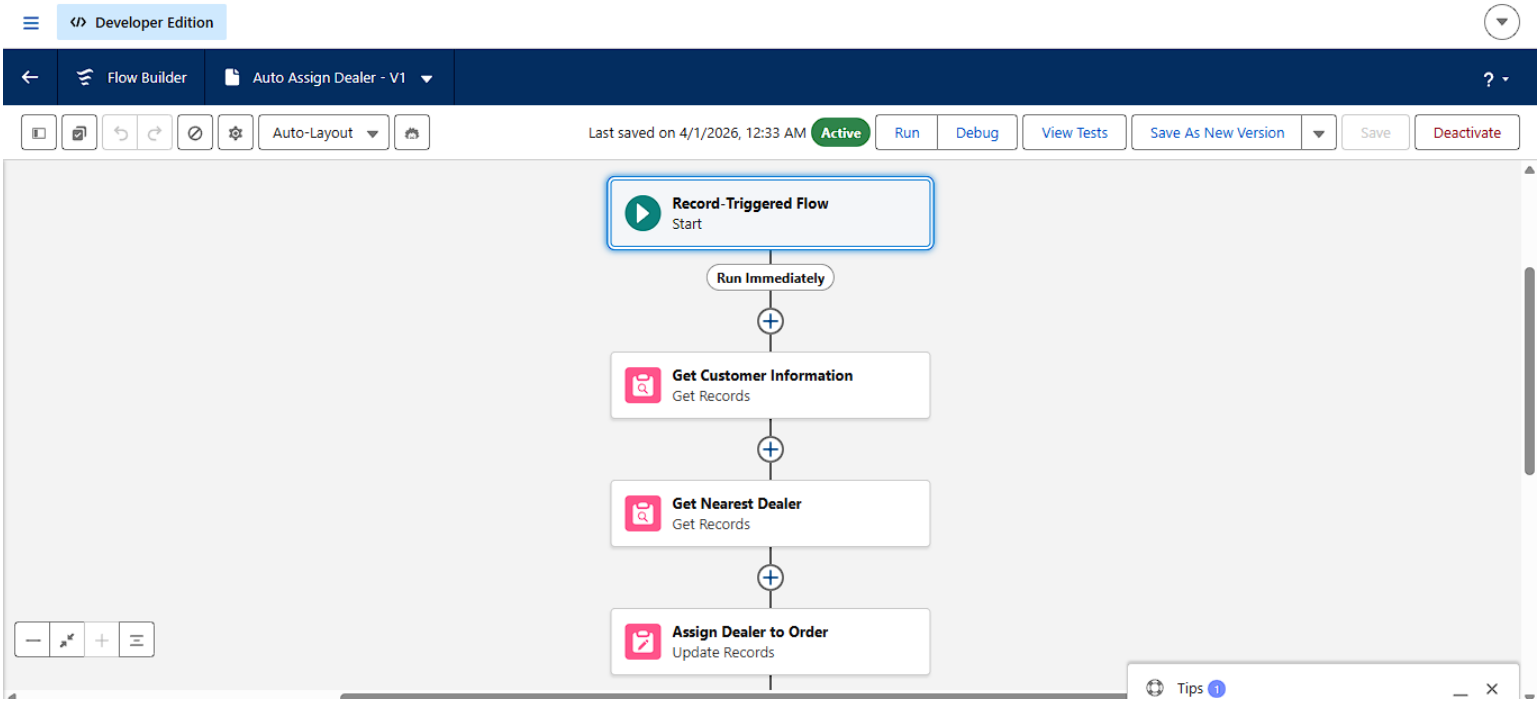


Step 8 : Click **+** → Select **Update Records**
Label : Assign Dealer to Order
*How to Find Records to Update and Set Their Values : Use the IDs and all field values from a record or record collection

Select Record(s) to Update : {!Get_Nearest_Dealer}



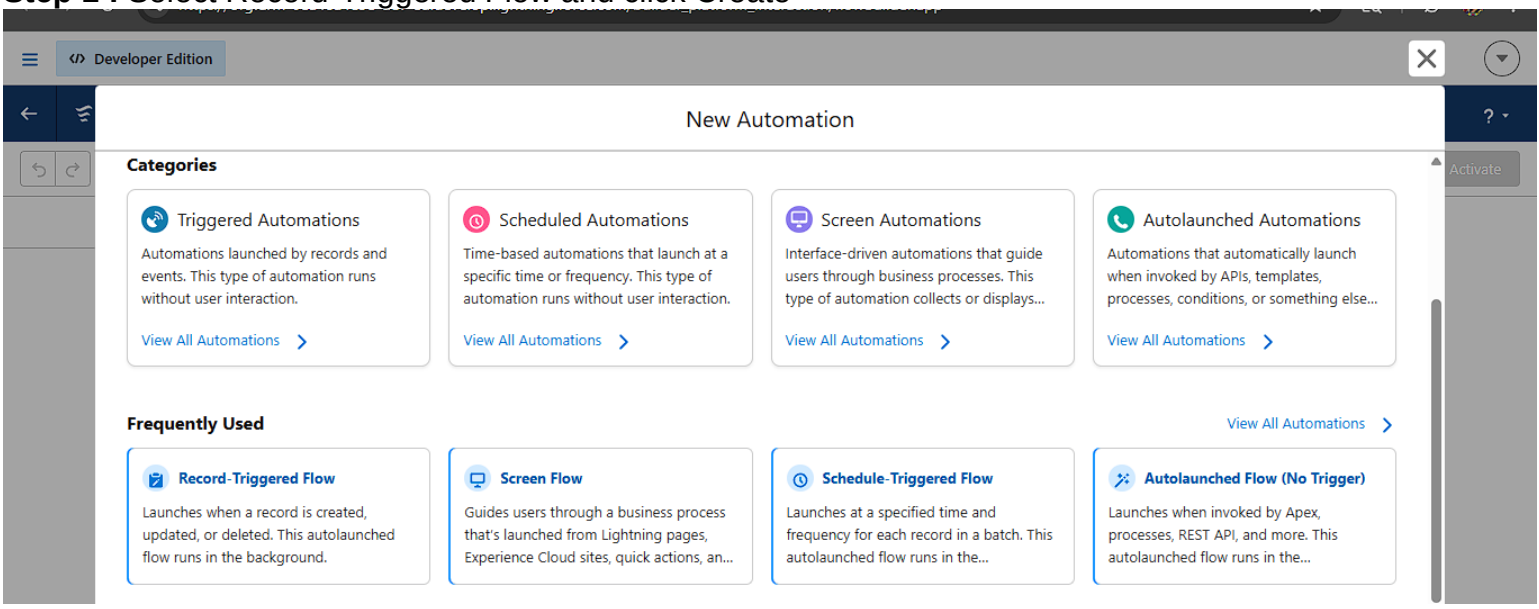
Step 9 : Click Save and Give label Name and Activate Flow.
Label Name : Auto Assign Dealer



Step 10 : Activate Flow

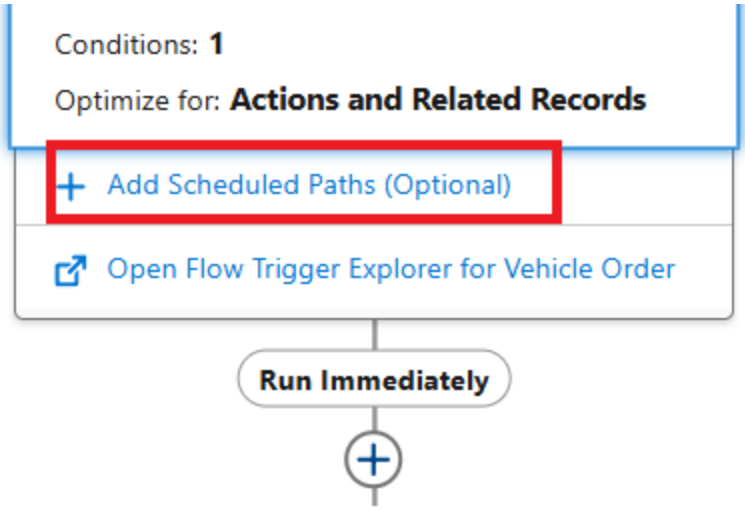
Activity 2: Record-Trigger Flow Creation

Step 1 : Select Record-Triggered Flow and click Create

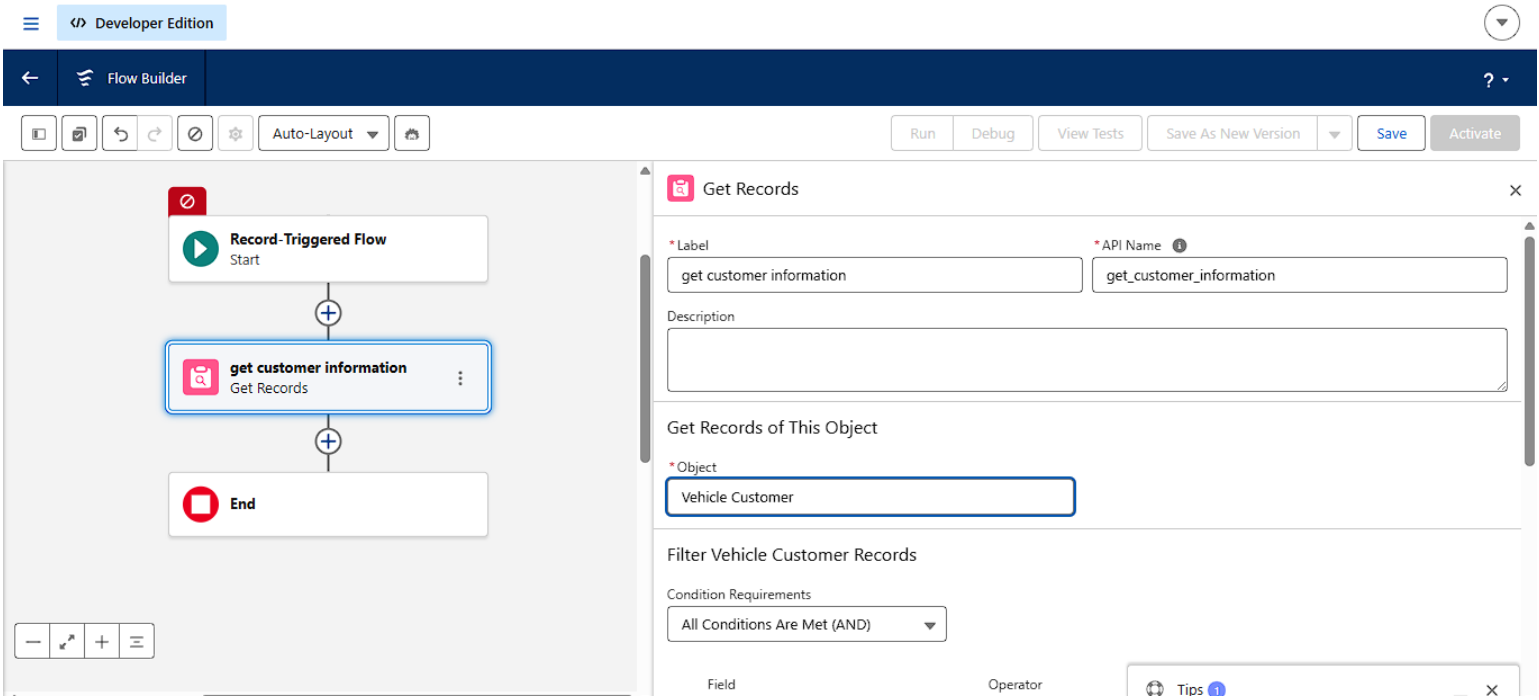


Step 2 : Select Vehicle Test Drive Object
Trigger the Flow When : A record is created or updated
Set Entry Condition :
All Conditions Are Met (AND)
Filed : Status__c
Operator : Equals
Value : Scheduled

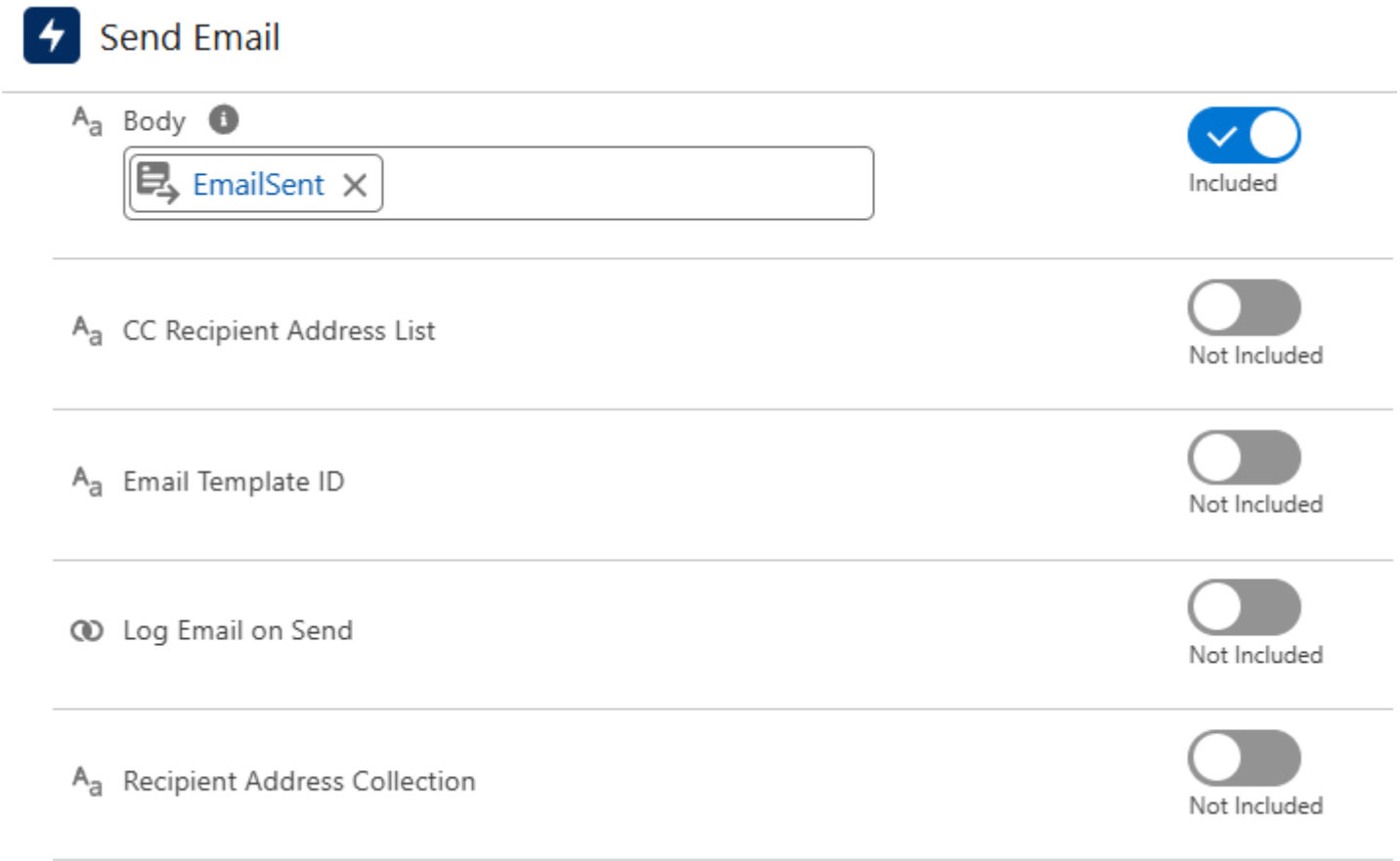
Step 3 : Click + Add Scheduled Paths (below the trigger).
Label: Reminder Before Test Drive.
Time Source: Test_Drive_Date__c
Offset Number: 1.
Offset Options: Days Before.
Click Done.



Step 4 :
Click + Add Element → Get Records.
Label: Get Customer Information.
Object: Vehicle_Customer__c.
Filter Conditions: Id = {!$Record.Customer__c}.
How Many Records to Store: Select Only the first record.
How to Store Record Data: Choose Automatically store all fields.

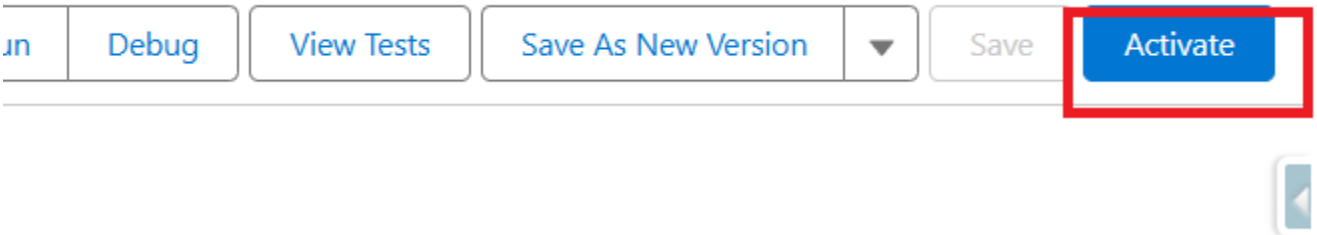


Step 5: Send Reminder Email
Click + Add Element → Action.
Action Type: Send Email
Label: Send Test Drive Reminder.
Subject: "Reminder: Your Test Drive is Tomorrow!".
Recipient Address List : {!Get_Customer_Information.Email__c}
Rich-Text-Formatted Body : True

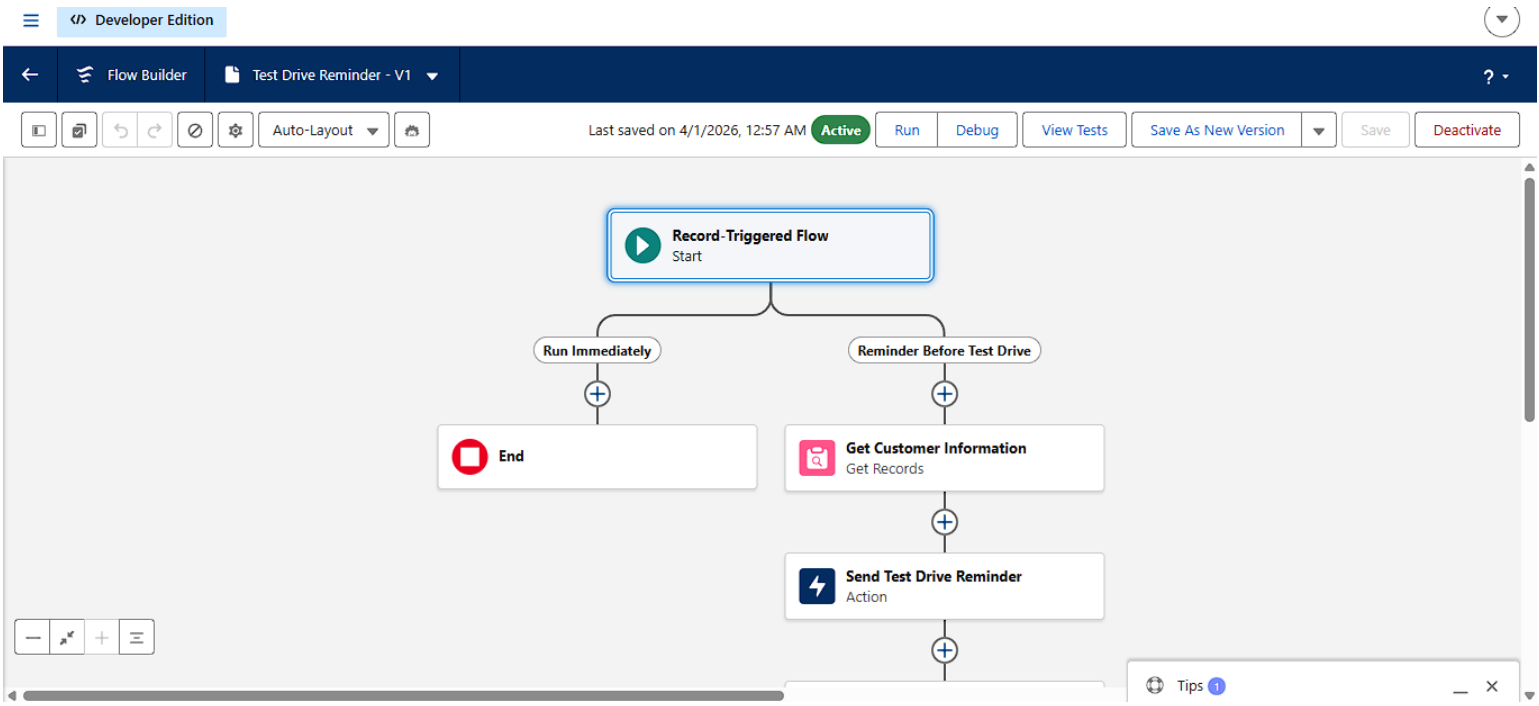


Step 6 : Click save
Label Name : Test Drive Reminder

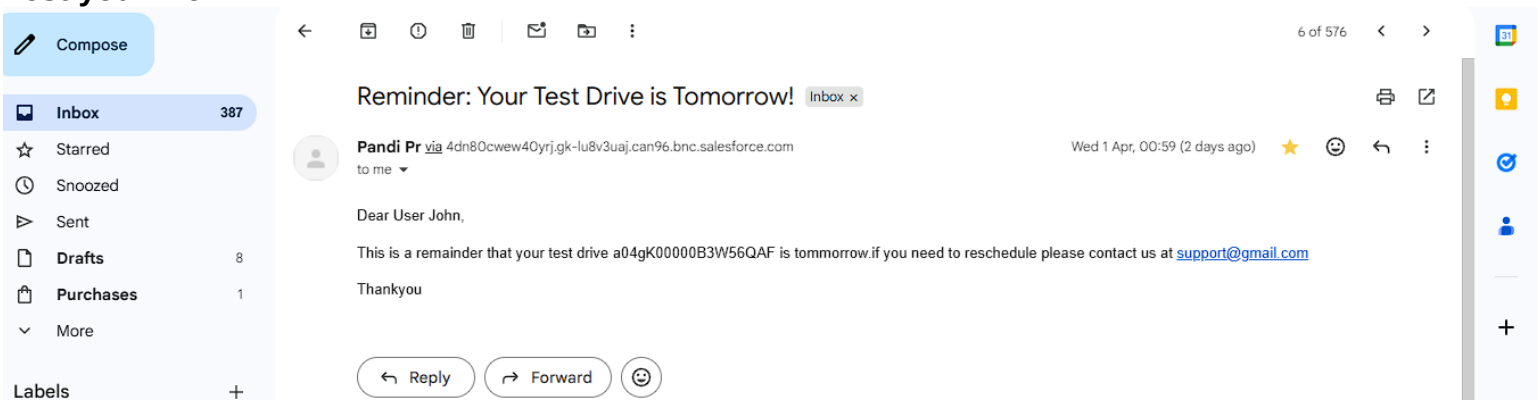
Step 7 : Activate Flow



Flow Page After Creation
:

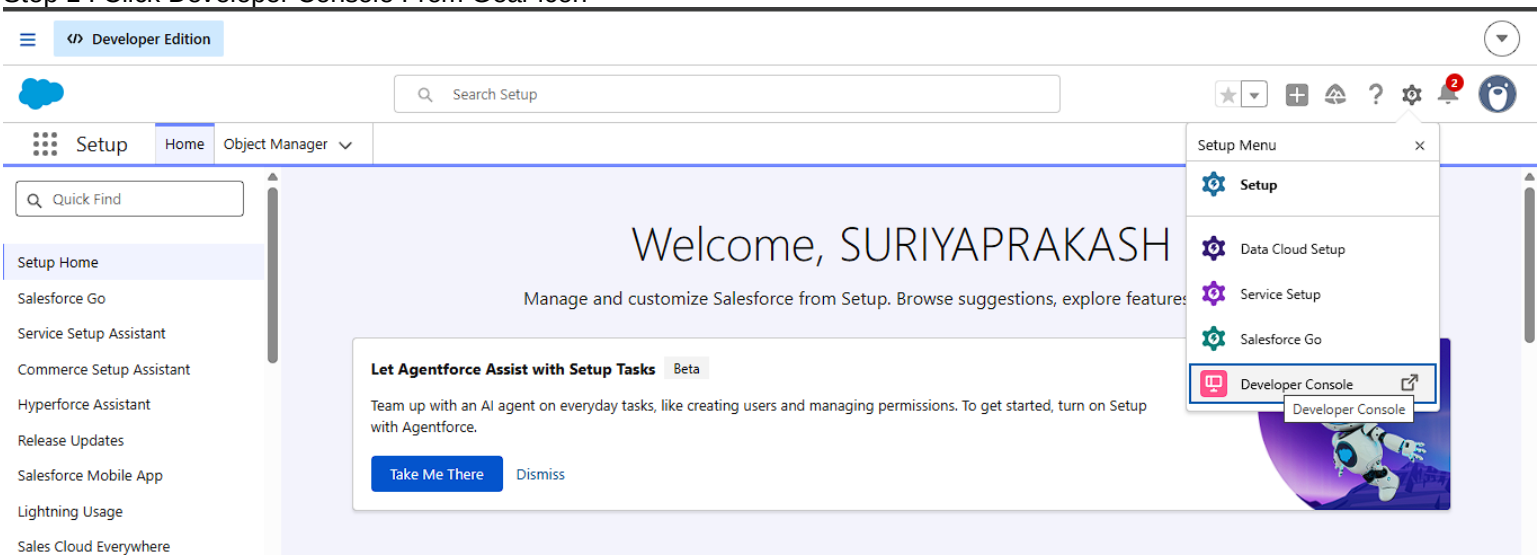


Test your Flow :

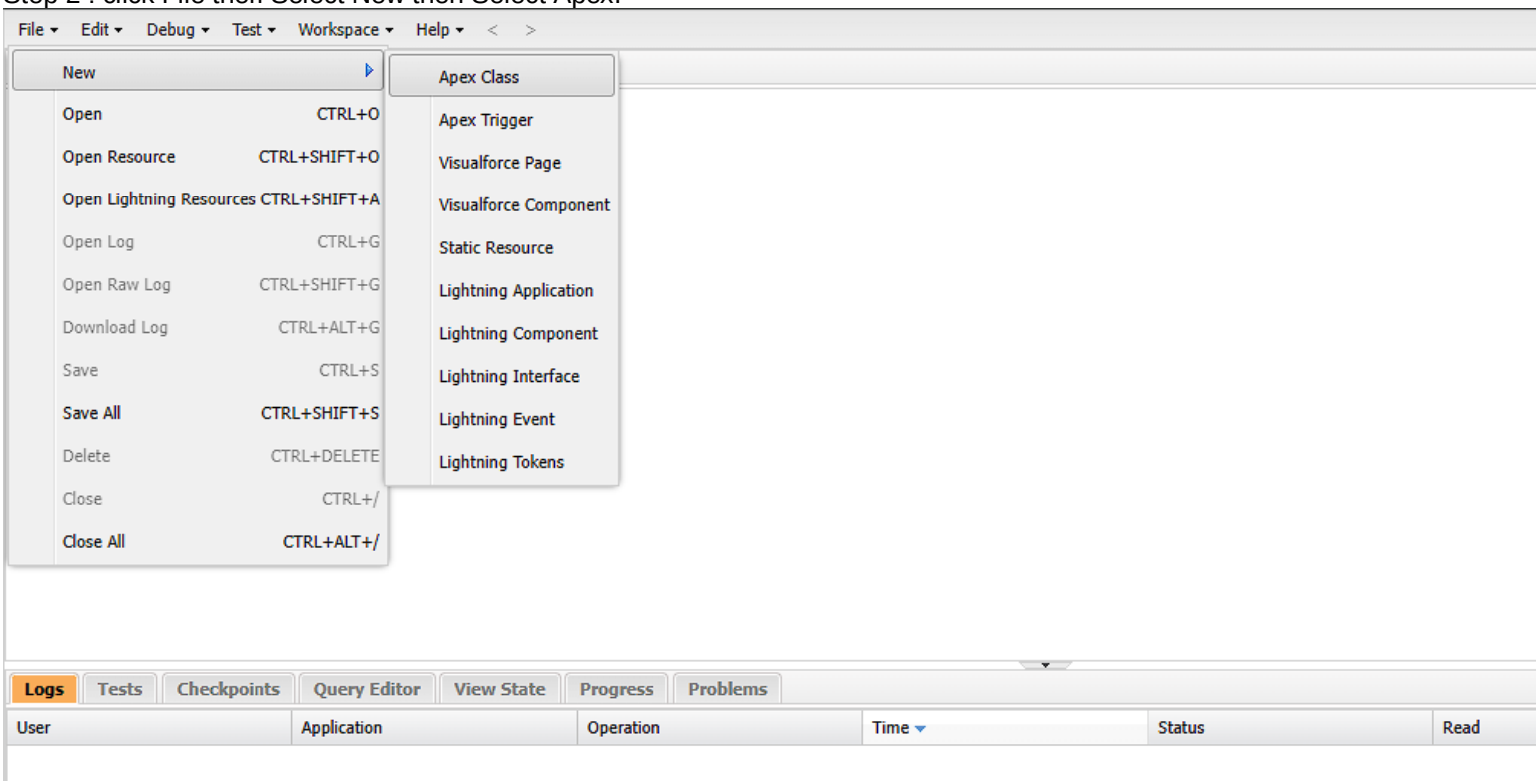


Milestone 7- Create Apex and Trigger Batch Jobs

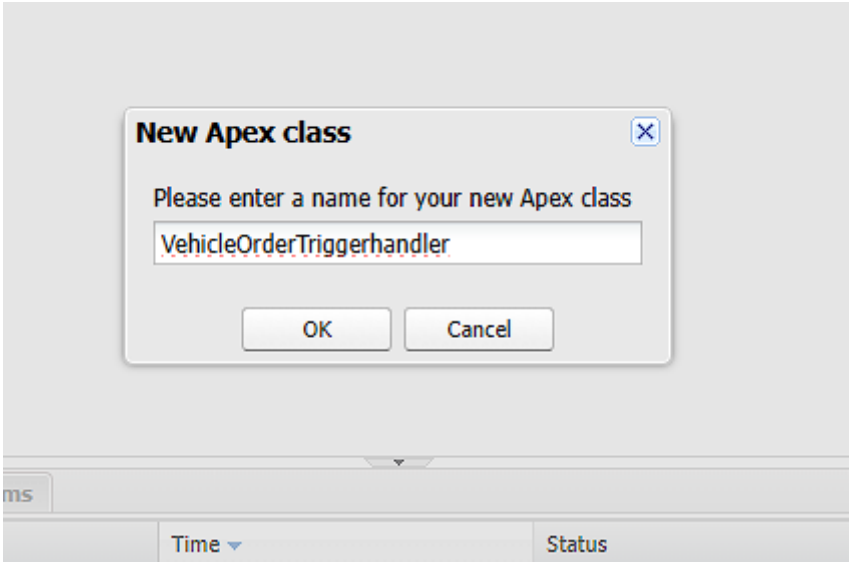
Step 1 : Click Developer Console From Gear icon



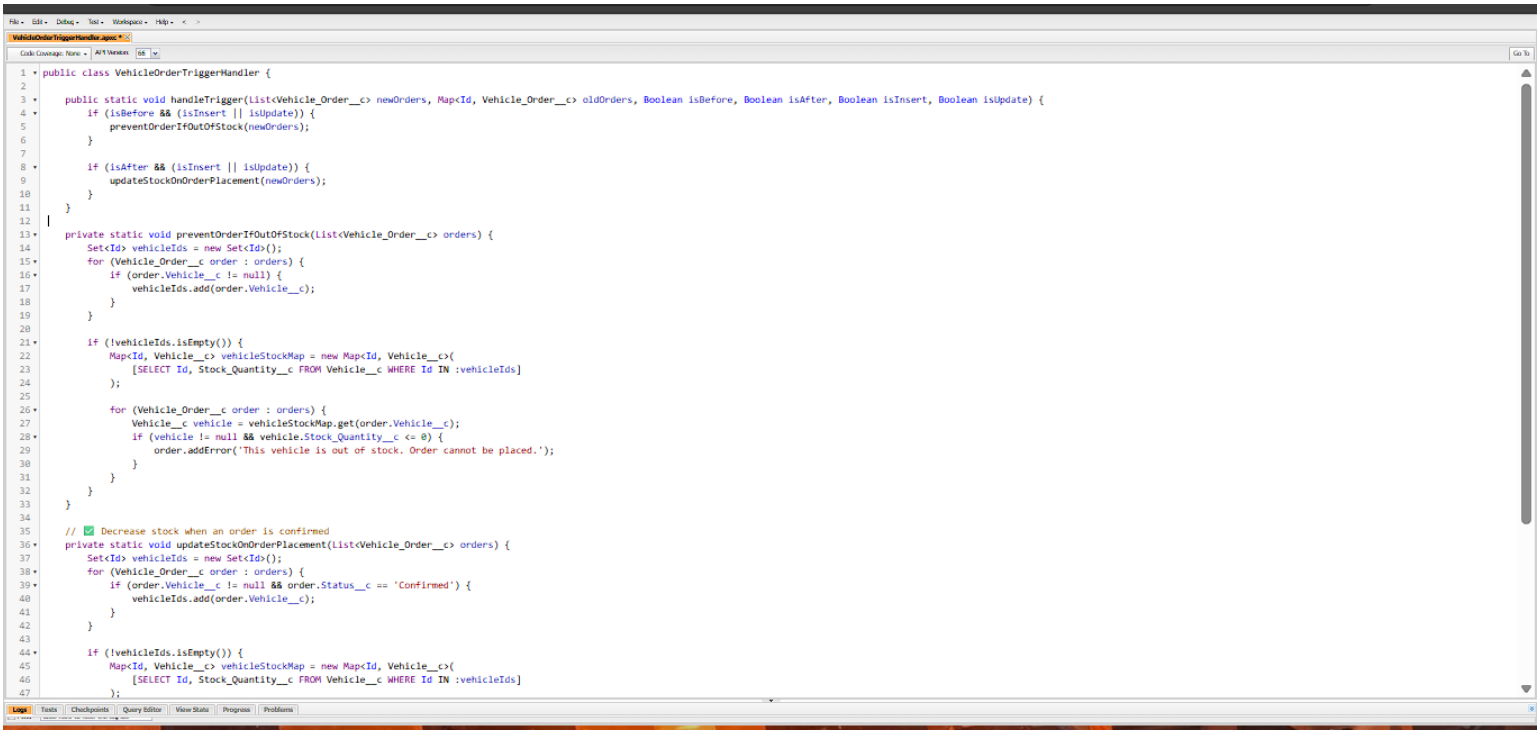
Step 2 : click File then Select New then Select Apex.



Step 3 : Give Name For Apex Class
Class Name : VehicleOrderTriggerhandler


New Apex class
Please enter a name for your new Apex class
VehicleOrderTriggerhandler
OK
Cancel
Time
Status

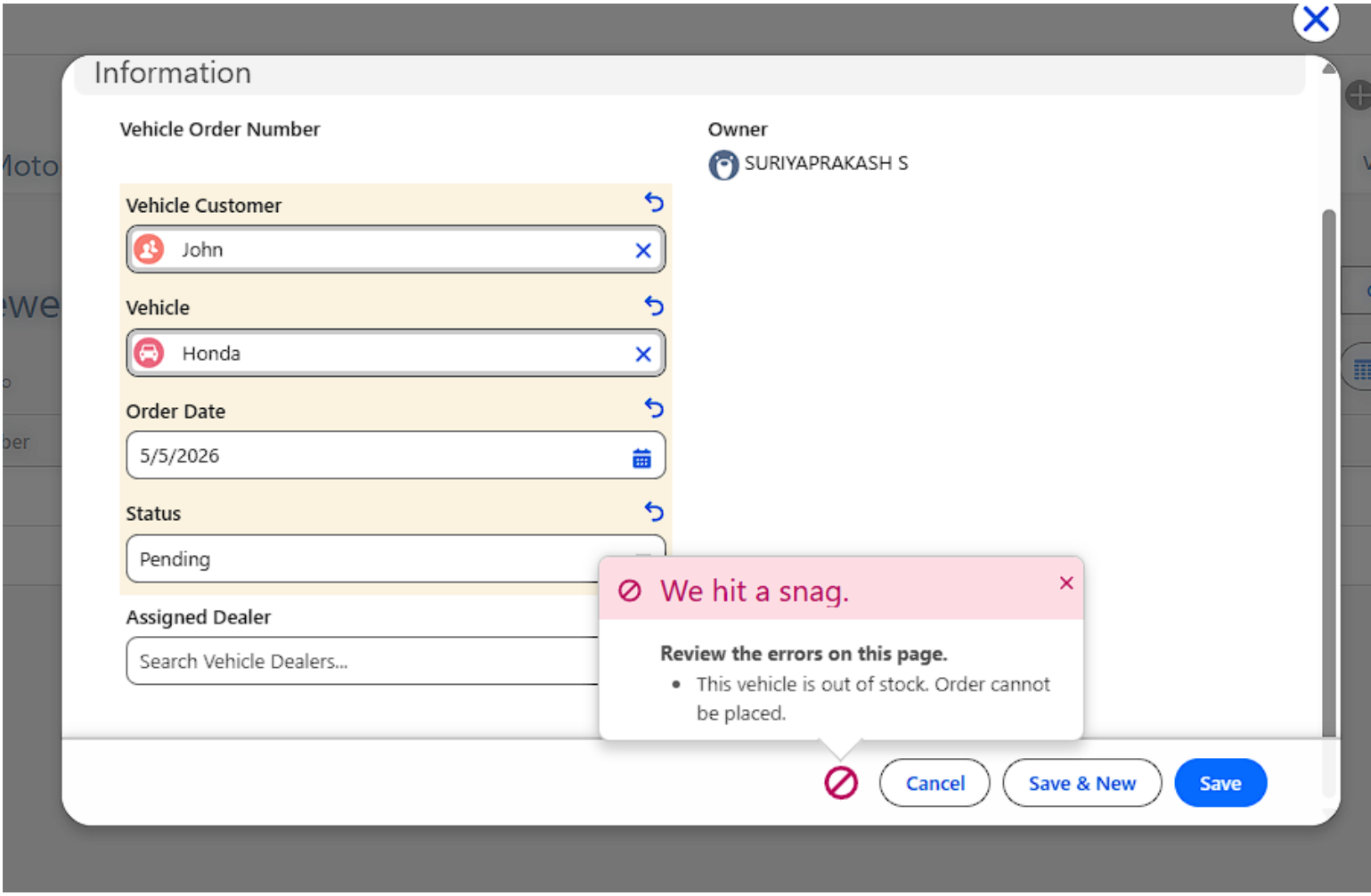
Step 4 : Write Apex Code



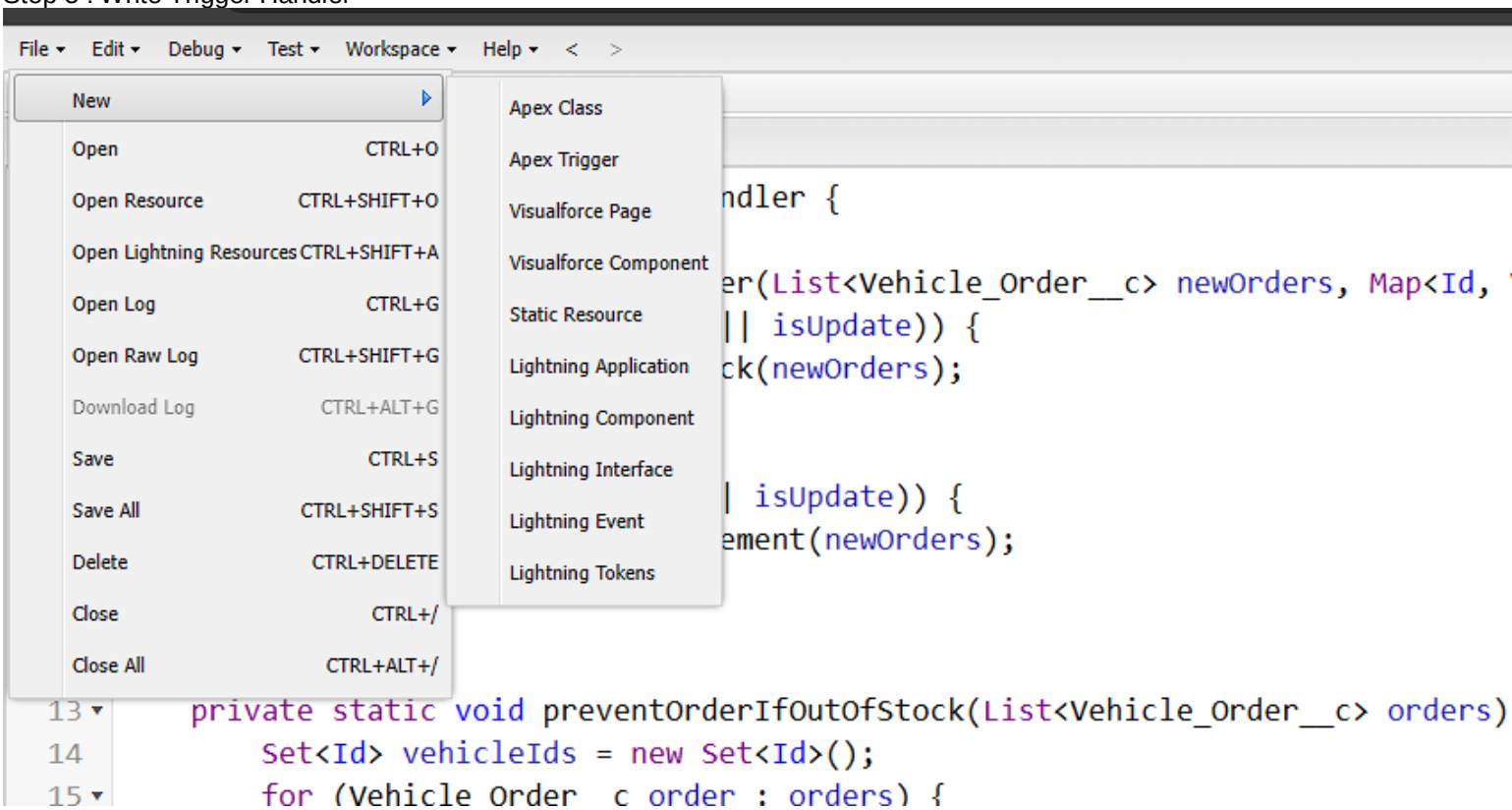
Source Code :

```
public class VehicleOrderTriggerHandler {
    public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id, Vehicle_Order__c> oldOrders, Boolean isBefore,
Boolean isAfter, Boolean isInsert, Boolean isUpdate) {
        if (isBefore) {
            if (isInsert || isUpdate) {
                preventOrderIfOutOfStock(newOrders);
            }
        }

        if (isAfter) {
            if (isInsert || isUpdate) {
                updateStockOnOrderPlacement(newOrders);
            }
        }
    }
    // Method to prevent orders when the vehicle is out of stock
    private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }
        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
            for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
                vehicleStockMap.put(vehicle.Id, vehicle);
            }
            for (Vehicle_Order__c order : orders) {
                if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                    Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                    if (vehicle.Stock_Quantity__c <= 0) {
                        order.addError('This vehicle is out of stock. Order cannot be placed.');
                    }
```

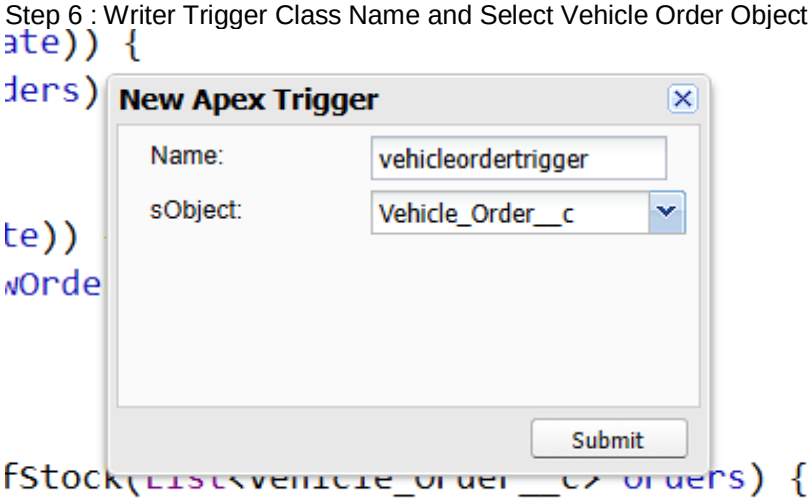
```
            }
        }
    }
}
// Method to update vehicle stock when an order is placed
private static void updateStockOnOrderPlacement(List<Vehicle_Order__c> orders) {
    Set<Id> vehicleIds = new Set<Id>();
    for (Vehicle_Order__c order : orders) {
        if (order.Vehicle__c != null && order.Status__c == 'Confirmed') {
            vehicleIds.add(order.Vehicle__c);
        }
    }
    if (!vehicleIds.isEmpty()) {
        Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
        for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
            vehicleStockMap.put(vehicle.Id, vehicle);
        }
        List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
        for (Vehicle_Order__c order : orders) {
            if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                if (vehicle.Stock_Quantity__c > 0) {
                    vehicle.Stock_Quantity__c -= 1;
                    vehiclesToUpdate.add(vehicle);
                }
            }
        }
        if (!vehiclesToUpdate.isEmpty()) {
            update vehiclesToUpdate;
        }
    }
}
}
```



Step 5 : Write Trigger Handler



Step 6 : Writer Trigger Class Name and Select Vehicle Order Object



Step 7 : Call Apex Class in Trigger Class

Source Code:

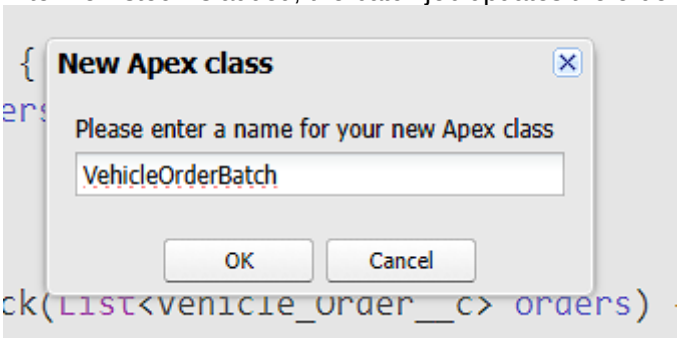
```
trigger VehicleOrderTrigger on Vehicle_Order__c (before insert,before update, after insert, after update) {
   VehicleOrderTriggerHandler.handleTrigger(Trigger.new, Trigger.oldMap, Trigger.isBefore,Trigger.isAfter, Trigger.isInsert, Trigger.isUpdate);
}
```

Step 8 : Create Batch Job

A customer places an order, but the vehicle is out of stock.

The order remains pending.

After new stock is added, the batch job updates the order to confirmed.



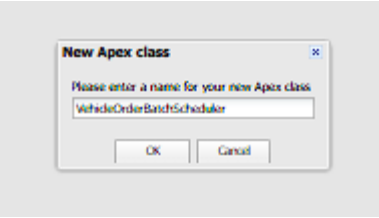
```apex
global class VehicleOrderBatch implements Database.Batchable<sObject> {

    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator([
            SELECT Id, Status__c, Vehicle__c FROM Vehicle_Order__c WHERE Status__c = 'Pending'
        ]);
    }

    global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orderList) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }

        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>(
                [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]
            );

            List<Vehicle_Order__c> ordersToUpdate = new List<Vehicle_Order__c>();
            List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();

            for (Vehicle_Order__c order : orderList) {
                Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                if (vehicle != null && vehicle.Stock_Quantity__c > 0) {
                    order.Status__c = 'Confirmed';
                    vehicle.Stock_Quantity__c -= 1;
                    ordersToUpdate.add(order);
                    vehiclesToUpdate.add(vehicle);
                }
            }

            if (!ordersToUpdate.isEmpty()) update ordersToUpdate;
            if (!vehiclesToUpdate.isEmpty()) update vehiclesToUpdate;
        }
    }

    global void finish(Database.BatchableContext bc) {
        System.debug('Vehicle order batch job completed.');
    }
}
```

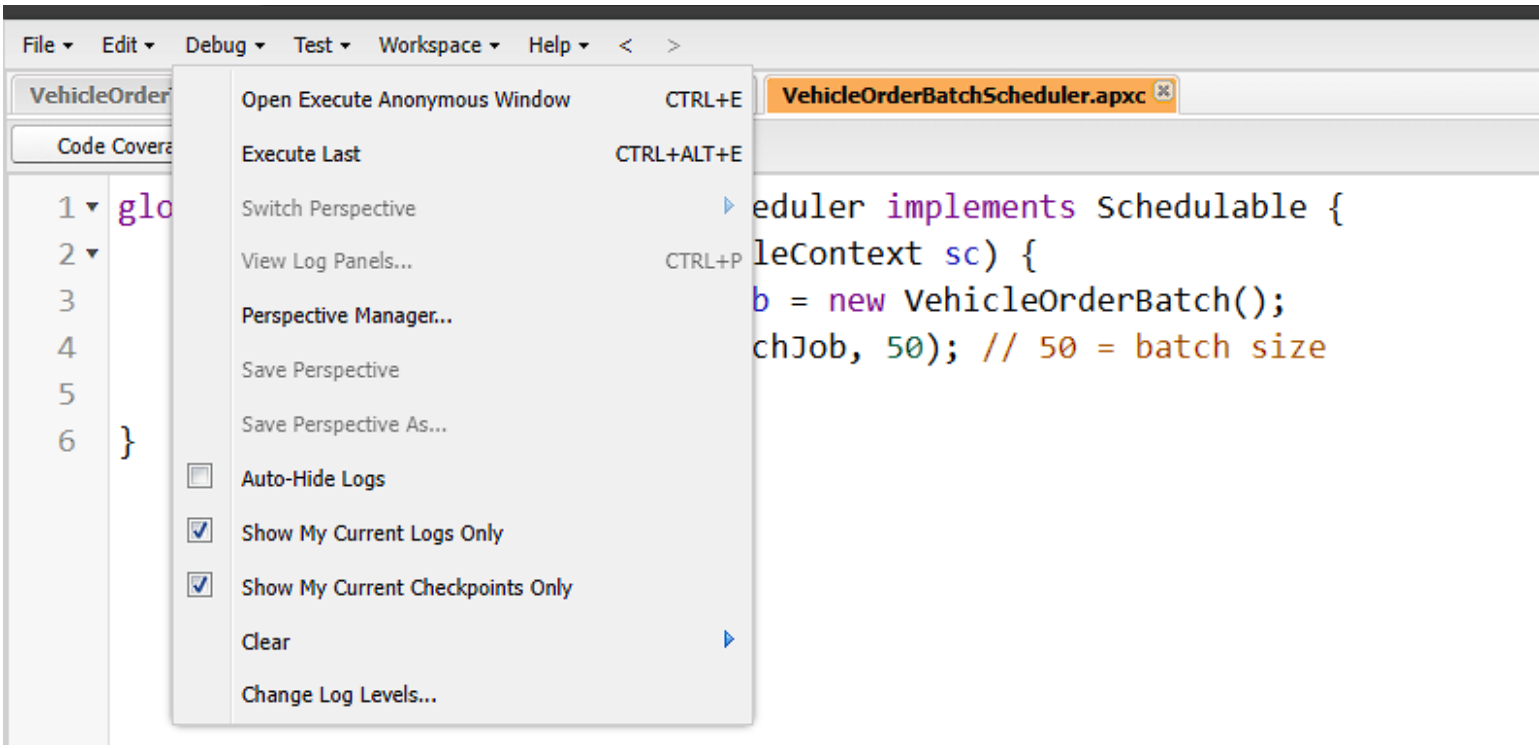
Source Code:

global class VehicleOrderBatch implements Database.Batchable<sObject> {
global Database.QueryLocator start (Database.BatchableContext bc) {
return Database.getQueryLocator([
SELECT Id, Status_c, Vehicle___c
FROM Vehicle_Order___c
WHERE Status__c
'Pending'
]);
}
global void execute(Database.BatchableContext bc, List<Vehicle_Order_c> orderList) {
Set<Id> vehicleIds = new Set<Id>();
for (Vehicle_Order_c order orderList) {
if (order.Vehicle_c != null) {
vehicleIds.add(order.Vehicle__c);
}
}
if (!vehicleIds.isEmpty()) {
Map<Id, Vehicle___c> vehicleStockMap = new Map<Id, Vehicle___c>();
for (Vehicle_c vehicle: [SELECT Id, Stock Quantity_c FROM Vehicle_c WHERE Id IN vehicleIds]) {
vehicleStockMap.put(vehicle. Id, vehicle);
}
List<Vehicle_Order_c> ordersToUpdate = new List<Vehicle_Order_c>(); List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
for (Vehicle_Order_c order orderList) {
if (vehicleStockMap.containsKey(order. Vehicle__c)) {

```
Vehicle_c vehicle = vehicleStockMap.get(order. Vehicle__c);
if (vehicle.Stock_Quantity___c > 0) {
order.Status___c = 'Confirmed';
vehicle.Stock_Quantity___c -= 1;
ordersToUpdate.add(order);
vehiclesToUpdate.add(vehicle);
}
}
}
if (!ordersToUpdate.isEmpty()) {
update ordersToUpdate;
}
if (!vehicles ToUpdate.isEmpty()) {
update vehiclesToUpdate;
}
}
global void finish(Database.BatchableContext bc) {
System.debug('Vehicle order batch job completed.');
}
}
```

Step 9 : Create Schedule Class then revoked Batch Class in Schedule Class



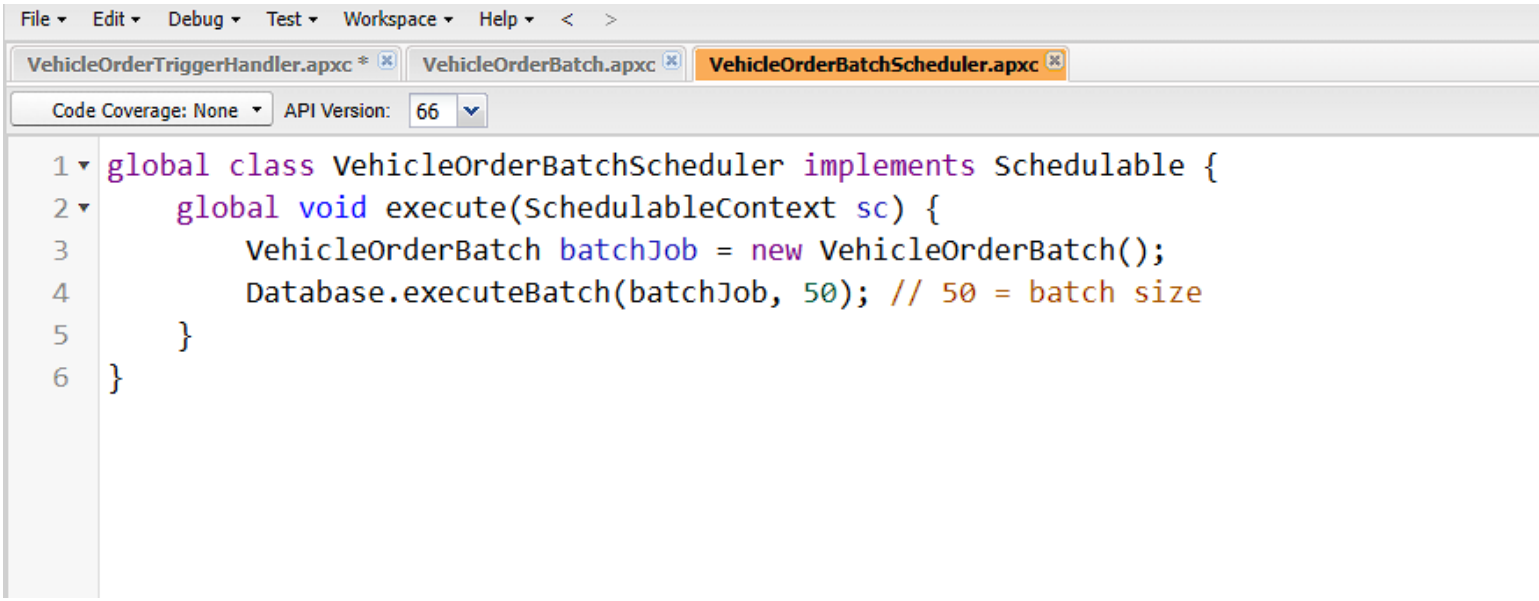


Source Code :

```
global class VehicleOrderBatchScheduler implements Schedulable {
```

```
    global void execute(SchedulableContext sc) {
        VehicleOrderBatch batchJob = new VehicleOrderBatch();
        Database.executeBatch(batchJob, 50); // 50 is the batch size
    }
}
```

Step 10 : S**chedule the Batch Job** To run the batch job **every night at midnight** :



Source Code :

```
String cronExp = '0 0 12 * * ?'; // Runs daily at 12:00 PM
System.schedule('Daily Vehicle Order Processing', cronExp, new VehicleOrderBatchScheduler());
```

Step 11 : You Can Check where this Schedule Job is Running

Milestone 8- Conclusion

In conclusion, the development of the WhatsNext Vision Motors CRM application represents a significant advancement in improving the company's operational efficiency and customer experience. By integrating vehicle, dealer, customer, and order data into a centralized system, the application streamlines the entire vehicle ordering process and ensures better data management.
The implementation of intelligent features such as automatic dealer suggestion based on customer location and real-time stock validation enhances convenience and prevents order errors. Automated order status updates provide clear and transparent communication to customers, improving trust and satisfaction.
Furthermore, the use of automation through flows and Apex reduces manual effort, minimizes errors, and increases overall productivity. Reports and dashboards enable data-driven decision-making, allowing the organization to respond effectively to business needs.
Overall, the system delivers a faster, more reliable, and customer-focused solution, helping WhatsNext Vision Motors strengthen its service quality and operational performance

1. FUNCTIONAL & PERFORMANCE TESTING

During the testing phase, screenshots were captured from the Salesforce Lightning interface to verify correct system behavior. These include:

- Record creation for Vehicle, Dealer, Customer, Order, Test Drive, and Service Request objects
- Validation rule testing for customer details such as phone number and required fields
- Verification of relationships between objects (Customer–Order, Vehicle–Order, Dealer–Vehicle)
- Testing of stock validation to ensure orders cannot be placed for out-of-stock vehicles
- Verification of automatic dealer suggestion based on customer location
- Testing of automated order status updates (Pending / Confirmed)
- Flow execution for automation processes such as order handling and updates
- Report and dashboard generation for orders, vehicles, and customer insights

---

Performance Testing

Performance testing ensured fast response time and system stability during concurrent usage.

- The system was tested for quick order processing and real-time stock validation
- Reports and dashboards were verified for fast loading and accurate data display
- Automation processes (Flows and Apex Batch) were tested for efficient execution
- The system maintained stability and performance even with multiple users accessing data simultaneously

2. ADVANTAGES & DISADVANTAGES

Advantages

- Centralized management of vehicle, dealer, customer, and order data
- Reduced manual work through automation of ordering and status updates
- Accurate order processing with real-time stock validation
- Improved customer experience with nearest dealer suggestion
- Enhanced operational efficiency using automation (Flows & Apex)
- Real-time analytics through reports and dashboards

Disadvantages

- Requires Salesforce license for full functionality
- Internet dependency for accessing the system
- Limited offline access
- Initial setup and configuration complexity

3. Conclusion:

The CRM application for WhatsNext Vision Motors successfully enhances the vehicle ordering process by integrating customer, dealer, vehicle, and order data into a unified system. The implementation of features such as automatic dealer suggestion, stock validation, and order status automation ensures accurate and efficient operations.

The system reduces manual effort, minimizes errors, and improves transparency in order management. Additionally, reports and dashboards enable data-driven decision-making, while automation improves overall productivity.

Overall, the solution delivers a reliable, efficient, and customer-centric system that strengthens business performance and customer satisfaction.

4. FUTURE SCOPE

- Mobile CRM application for on-the-go access
- AI-based vehicle demand and sales prediction
- Integration with WhatsApp/SMS for customer notifications
- Advanced inventory and stock management system
- Multi-branch and multi-location dealer support
- Enhanced customer profiling and personalization

5. APPENDIX

- **Source Code:** Salesforce Configuration, Flows, Apex & Batch Classes
- **Dataset:** Vehicle, Customer, Dealer, and Order Records
- **Tools Used:** Salesforce Developer Org

Thank You